



 slington college
(इस्लिंग्टन कलेज)

Module Code & Module Title

CS6P05NI Final Year Project

Assessment Weightage & Type

25% FYP Interim Report

Semester

2024-25 Autumn/Spring

Project Title: TaskHive (SME- freelancer matchmaking platform)

Student Name: Simanka Paudel

London Met ID: 23050271

College ID: NP01CP4A230098

Internal Supervisor: Mr Saurabh Adhikari

External Supervisor: Mr Bibek Maharjan

Assignment Due Date: Tuesday, 30 December 2025

Assignment Submission Date: Tuesday, 30 December 2025

Word Count (Where required): 3500

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Acknowledgement

I would like to express my sincere gratitude to the Faculty of Computing at Islington College for giving me the opportunity to undertake this project and for their guidance throughout the development of this proposal. I am especially thankful to my supervisors, whose advice and feedback provided me with a clear understanding of the expected standards and structure for this submission.

I would also like to acknowledge the valuable resources and learning materials provided by the college, which greatly influenced the concept and direction of my project. Finally, I am grateful to my friends for their support and for engaging in discussions about the project idea, which helped refine and improve the concept.

Abstract

It is clear that Small and Medium Enterprises (SMEs) in Nepal face significant challenges in accessing affordable IT services, while many student developers and freelancers struggle to find dependable project opportunities. This proposal introduces the Freelancer–SME Matchmaking and Escrow-Based Project Management System, a web-based platform designed to connect SMEs with skilled freelancers in a secure and structured environment. Key features of the system include project posting, proposal submission, secure escrow payments via local gateways, messaging, KYC verification, and a review system. By integrating these elements, the platform aims to build trust, support effective project collaboration, provide cost-effective digital solutions for SMEs, and assist both emerging and existing freelancers.

Table of Contents

1	Introduction	1
1.1	Project Description	1
1.2	Problem Scenario.....	2
1.2.1	Local Scenario.....	2
1.2.2	Global Scenario	2
1.3	Project as a solution.....	3
1.3.1	Matchmaking of SMEs and Freelancers.....	3
1.3.2	Escrow Payment System.....	3
1.3.3	Cost and Timeline Estimation	3
1.4	Aims and Objectives	3
1.4.1	Aim	3
1.4.2	Academic Objectives	3
1.4.3	Project Objectives.....	3
2	Background.....	4
2.1	About the targeted user.....	4
2.2	Understanding the solution	5
2.3	Similar Projects	5
2.3.1	Review of similar projects	5
2.3.2	Comparison between the similarity of the systems.....	5
2.3.3	Comparison analysis of similar projects with the proposed project.....	6
2.4	Methodology.....	7
2.4.1	Considered Methodology.....	7
2.4.2	Selected Methodology: Agile Scrum.....	7
3	Work till date	9
3.1	Requirement Gathering.....	9
3.1.1	SRS	10
3.1.2	Product backlog.....	10
3.1.3	Pre survey	14
3.2	System Design.....	17
3.2.1	Use case.....	17
3.2.2	Entity Relationship Diagram	18

3.2.3	Class Diagram	18
3.2.4	Data Flow Diagram	19
3.3	Sprint 1 / Prototype 1 – Login and Register	21
3.3.1	Sprint Planning	21
3.3.2	Design	21
3.3.3	Development	29
3.3.4	Sprint Review	31
3.3.5	Sprint retrospective.....	31
3.3.6	Velocity Chart	32
4	Progress Analysis	32
4.1	Gantt chart	32
4.2	Progress Table.....	34
4.3	Analysis.....	35
5	Further works	35
5.1	Sprint 2: Project Posting and Bidding Module	35
5.2	Sprint 3: Escrow and Payment Management	35
5.3	Sprint 4: Communication, File Sharing, and Review System	36
5.4	Sprint 5: Administrative Dashboard and KYC Verification	36
5.5	Sprint 6: AI-Based Cost and Timeline Estimation.....	36
5.6	Sprint 7: System Integration, Testing, and Optimization	36
6	Conclusion	36
7	References.....	38
8	Appendix	39
8.1	Problem Scenario.....	39
8.1.1	Local Scenario	39
8.1.2	Global Scenario	40
8.2	Background	40
8.2.1	About the targeted users	40
8.2.2	Understanding the solution	40
8.3	Review of similar projects	41
8.3.1	Project 1- Upwork	41
8.3.2	Project 2 – Fiverr	42

8.3.3	Project 3 – Freelancer.com.....	43
8.3.4	Project 4 – Toptal	44
8.3.5	Project 5 – Guru	44
8.4	Considered Methodologies.....	45
8.4.1	Methodology 1 – Waterfall Model	45
8.4.2	Methodology 2 – V-Model.....	46
8.4.3	Methodology 3 – Spiral Model	46
8.4.4	Methodology 4 – Prototyping Model	46
8.4.5	Methodology 5 – Agile Methodology.....	46
8.5	Requirement Gathering.....	47
8.6	SRS.....	47
8.6.1	Introduction.....	47
8.6.2	Overall Description	48
8.6.3	Functional Requirements.....	49
8.6.4	Non-Functional Requirements	65
8.6.5	External Interface Requirements	67
8.6.6	User Interface	67
8.6.7	Hardware Interface	67
8.6.8	Software Interface	67
8.7	Sprint 1 / Prototype 1 – Login and Register	67
8.7.1	Sprint Planning	67
8.7.2	Design	68
8.8	Pre-survey response	80
8.9	Development.....	91
8.9.1	Frontend	91
8.9.2	Backend.....	100
8.10	Progress Analysis	108
8.10.1	Analysis.....	108
8.11	Future Works	108
8.11.1	Sprint 2: Project Posting and Bidding Module.....	108
8.11.2	Sprint 3: Escrow and Payment Management	109
8.11.3	Sprint 4: Communication, File Sharing, and Review System	110

8.11.4	Sprint 5: Administrative Dashboard and KYC Verification	111
8.11.5	Sprint 6: AI-Based Cost and Timeline Estimation.....	112
8.11.6	Sprint 7: System Integration, Testing, and Optimization.....	113

Table of figures

Figure 1: Comparison chart.....	6
Figure 2: Scrum artifacts (ATLASSIAN, 2024)	9
Figure 3:Survey response	15
Figure 4: Pre survey question 1 and 2 response	16
Figure 5: Use case diagram	17
Figure 6: Erd diagram.....	18
Figure 7: Class diagram	19
Figure 8: Context level diagram	20
Figure 9: DFD 1.....	20
Figure 10: DFD 2.....	21
Figure 11: Login activity diagram	22
Figure 12: Register activity diagram.....	23
Figure 13: Registration sequence diagram.....	24
Figure 14: Login sequence diagram.....	25
Figure 15: Login wireframe.....	26
Figure 16: Register Wireframe	27
Figure 17: Registration UI mockup.....	28
Figure 18: Login Mockup.....	29
Figure 19: Login UI.....	30
Figure 20: Register UI	31
Figure 21: Velocity chart.....	32
Figure 22: Gantt chart	32
Figure 23: Gantt chart sprint 1.....	33
Figure 24: Gantt chart sprint 2.....	33
Figure 25: Gantt chart sprint 3 and 4.....	33
Figure 26: Gantt chart sprint 5 and 6.....	33
Figure 27: Testing and Documentation gantt chart	34
Figure 29: Upwork.....	42
Figure 30: Fiverr	43
Figure 31: Freelancer.com	43
Figure 32: Toptal	44
Figure 33: Guru	45
Figure 34: user class and characteristics	48
Figure 45: Pre survey question 3 and 4 response.....	80
Figure 46: Pre survey question 5 and 6 response.....	81
Figure 47: Pre survey question 7 and 8 response.....	82
Figure 48: Pre survey question 9 and 10 response.....	83
Figure 49: Pre survey question 11 and 12 response.....	84
Figure 50: Pre survey question 13 and 14 response.....	85
Figure 51: Pre survey question 15 and 16 response.....	86
Figure 52: Pre survey question 17 and 18 response.....	87

Figure 53: Pre survey question 19 and 20 response	88
Figure 54: Pre survey question 21 and 22 response	89
Figure 55: Pre survey question 23 and 24 response	90
Figure 56: Pre survey question 25 response	91
Figure 57: Login.jsx	92
Figure 58: Login.jsx	93
Figure 59: Login.css	94
Figure 60: Login.css	95
Figure 61: Register.jsx	96
Figure 62: Register.jsx	96
Figure 63: Register.jsx	97
Figure 64: Register.css	98
Figure 65: Register.css	99
Figure 66: Register.css	100
Figure 67: User model	101
Figure 68: User controller	101
Figure 69: User controller	102
Figure 70: User Controller	103
Figure 71: User Controller	104
Figure 72: Auth Middleware	105
Figure 73: Auth Route	106
Figure 74: Auth Route	107
Figure 75: Config	107

Tables of table

Table 1: Product Backlog	14
Table 2: Progress table	35
Table 3: Functional Requirement of registration.....	51
Table 4: Login functional requirements	52
Table 5: Profile Management functional requirements	54
Table 6: Project Management SRS	56
Table 7: Browse and Apply SRS	58
Table 8: Bidding and matchmaking SRS.....	59
Table 9: Escrow-Based Payment System SRS	60
Table 10: Communication Module SRS	61
Table 11: Rating and review SRS	62
Table 12: Admin Dashboard SRS	64
Table 13: AI-based estimation SRS	65
Table 14: Non-functional requirement – reliability	66
Table 15: Non-functional requirement – Security	66
Table 16: Non-Functional requirement – performance	67

1 Introduction

In Nepal, small and medium-sized enterprises (SMEs) account for roughly 95% of all registered businesses and play a crucial role in generating employment, driving economic growth, and strengthening value chains. As these businesses increasingly move toward digital operations, the need for cost-effective IT solutions—such as website development, digital branding, marketing tools, and software—has grown. Yet, many SMEs struggle to work with professional IT agencies because of high expenses, lengthy project timelines, and limited transparency.

. (FRIEDRICH NAUMANN FOUNDATION, 2024)

At the same time, many student developers and freelancers in Nepal possess strong technical skills but lack organized channels to connect with SMEs. According to the ICT Council Nepal (2024), over 70,000 ICT students graduate or enter the workforce each year, yet many struggle to secure employment due to limited exposure and the absence of formal freelance platforms tailored to Nepal's context.

This mismatch between SME demands and the available talent highlights the need for a localized, secure, and transparent digital marketplace. Such a platform should allow SMEs to post projects, receive bids, communicate efficiently, and conduct transactions safely, while enabling freelancers to find verified clients, submit competitive bids, and receive guaranteed payments.

1.1 Project Description

This project aims to create a Freelancer–SME Matchmaking Platform that links small and medium enterprises with student freelancers within a secure and transparent digital environment. The platform enables SMEs to post project requirements, evaluate freelancer profiles, compare bids, and manage their projects effectively. Meanwhile, freelancers can showcase their skills, apply for relevant projects, interact with clients, and gain valuable professional experience.

The system incorporates role-based access for freelancers, SMEs, and administrators. Its main features include secure login, project management, communication tools, and administrative oversight. By focusing on affordability,

simplicity, and user-friendliness, the platform seeks to promote digital adoption among SMEs while providing meaningful opportunities for students.

1.2 Problem Scenario

SMEs in Nepal play a vital role in driving economic growth, but many face challenges such as limited funding, high costs for development, and unreliable digital services. At the same time, numerous IT students and entry-level freelancers have the skills and enthusiasm to contribute, yet they lack opportunities for practical project experience. While SMEs require affordable digital support and students need hands-on experience, there is rarely a structured way for them to connect.

One key reason for this disconnect is the common misconception about the IT field. Many students pursue IT due to external pressure or trends, and families often associate IT only with device repairs. This results in graduates who have theoretical knowledge but little practical experience, frustrating employers seeking job-ready talent. Meanwhile, SMEs are reluctant to adopt digital solutions because professional services are costly, slow, and often opaque.

Consequently, SMEs lose access to reliable and affordable digital assistance, while students miss chances to earn, learn, and build their portfolios. This highlights the need for a secure, locally-focused platform that links SMEs with student freelancers. Such a system would provide SMEs with verified talent and offer students real projects, fair compensation, and guided workflows—strengthening Nepal’s digital ecosystem and benefiting both parties.

. (The Himalayan, 2025)

1.2.1 Local Scenario

[Read More....](#)

1.2.2 Global Scenario

[Read More....](#)

1.3 Project as a solution

1.3.1 Matchmaking of SMEs and Freelancers

- SMEs post projects with skills, experience, budget, and deadlines.
- Freelancers can filter and apply for projects
- SMEs can compare proposals using a simple rating system
- Promotes transparency and better hiring decisions.

1.3.2 Escrow Payment System

- SMEs deposit project funds into escrow
- Payments released only after SME approval
- Admin manages disputes

1.3.3 Cost and Timeline Estimation

- Suggests budget range and project timeline based on historical data
- Warns SMEs if proposed budget or timeline is unrealistic

1.4 Aims and Objectives

1.4.1 Aim

The goal is to develop a secure, automated web platform that links freelancers with SMEs through organized matchmaking and bidding, using escrow payments to guarantee trust, transparency, and the successful delivery of projects.

1.4.2 Academic Objectives

- To apply fundamental software engineering principles by developing a functional, real-world system
- To gather and document system requirements accurately, adhering to standard SRS guidelines
- To represent system behavior and structure through UML diagrams, including use case, sequence, and activity diagrams

- To create a full-stack web application using modern development tools and frameworks
- To design databases effectively and ensure efficient data management
- To implement secure user authentication and role-based access controls
- To strengthen skills in testing, debugging, and preparing clear technical documentation

1.4.3 Project Objectives

- To provide SMEs with reliable and affordable IT services
- To offer structured and equitable work opportunities for student freelancers and early-career developers
- To implement a secure registration system with role-based login access
- To enable clear project listings, competitive bidding, and efficient communication
- To help freelancers gain hands-on experience and build strong portfolios
- To ensure data privacy and secure management of all system transactions
- To encourage digital adoption and innovation among local SME communities

2 Background

2.1 About the targeted user

The proposed platform is designed for Small and Medium Enterprises (SMEs) and freelancers, with particular emphasis on IT students and individuals at the early stages of their careers. SMEs contribute significantly to economic growth and job creation; however, many operate under financial and technical constraints. Due to these limitations, accessing dependable and cost-effective IT services—such as web development, software support, digital marketing, system enhancement, and technical assistance—often becomes a challenge. Employing full-time IT professionals is frequently impractical because of high salary requirements and long-term employment obligations.

[Read More....](#)

2.2 Understanding the solution

The proposed system is a web-based platform that connects freelancers with Small and Medium Enterprises (SMEs) to support effective and organized collaboration between both parties. The platform allows SMEs to post detailed project requirements, define budgets, and set expected timelines, while freelancers can explore available opportunities and apply or place bids according to their skills and preferences.

[Read More....](#)

2.3 Similar Projects

2.3.1 Review of similar projects

2.3.1.1 Project 1- Upwork

[Read More....](#)

2.3.1.2 Project 2 – Fiverr

[Read More...](#)

2.3.1.3 Project 3 – Freelancer.com

[Read More....](#)

2.3.1.4 Project 4 – Toptal

[Read More....](#)

2.3.1.5 Project 5 – Guru

[Read More....](#)

2.3.2 Comparison between the similarity of the systems

Features	Project 1 (Upwork)	Project 2 (Fiverr)	Project 3 (Freelancer)	Project 4 (Toptal)	Project 5 (Guru)	Project 6 (Proposed System)

Target Users	Global professionals	Global freelancers	Global freelancers	Elite global professionals	Global freelancers and employers	Local SME's And Students
Beginner Friendly	No	Moderate	Low	Low	Moderate	Yes
SME focus	Moderate	Low	Moderate	Moderate	Moderate	High
Local Relevance	Low	Low	Low	Low	Moderate	High
Secure Authentication	Yes	Yes	Yes	Yes	Yes	Yes

Figure 1: Comparison chart

2.3.3 Comparison analysis of similar projects with the proposed project

Global freelance platforms like Upwork, Fiverr, Freelancer.com, Toptal, and Guru mainly serve an international audience, which creates intense competition, high service fees, and limited opportunities for beginners or student freelancers from developing countries. In contrast, the proposed system is designed specifically for local SMEs and entry-level IT professionals, making it easier for novices to access projects and gain practical experience.

The platform directly addresses local challenges such as cost, transparency, and accessibility, while fostering learning and mentorship. Unlike global marketplaces that prioritize scale and revenue, this system focuses on skill-building, reliability, and long-term professional growth. By simplifying project posting, bidding, and communication, it ensures SMEs can obtain trustworthy IT services, while freelancers receive fair opportunities to earn and develop their portfolios, supporting a sustainable and thriving local digital ecosystem.

2.4 Methodology

2.4.1 Considered Methodology

2.4.1.1 Methodology 1 – Waterfall Model

[Learn More..](#)

2.4.1.2 Methodology 2 – V-Model

[Learn More....](#)

2.4.1.3 Methodology 3 – Spiral Model

[Learn More....](#)

2.4.1.4 Methodology 4 – Prototyping Model

[Learn More....](#)

2.4.1.5 Methodology 5 – Agile Methodology

[Learn More....](#)

2.4.2 Selected Methodology: Agile Scrum

Scrum is a popular Agile methodology that assists teams in handling complex projects by breaking work into short, iterative cycles called sprints. Its primary aim is to enhance collaboration, increase transparency, and promote continuous improvement.

(ATLASSIAN, 2024).

Scrum artifacts:

- **Product** **Backlog**
The product backlog is a continually updated, prioritized list of user stories, features, tasks, and bugs required for the final product. Each item is defined clearly enough for planning, and the team estimates them during backlog refinement to help the product owner determine priorities.
- **Sprint** **Backlog**
The sprint backlog contains the user stories and tasks selected for a specific sprint. It is created during sprint planning, with tasks broken down and tracked on a board

(To-Do, Doing, Verify, Done). The backlog is updated as work progresses, serving as the team's short-term action plan.

- **Product** **Vision**

The product vision defines the long-term purpose of the product and the value it intends to provide. It gives the team a clear understanding of the ultimate goal. The product owner develops this vision based on user needs, market research, and business objectives.

- **Sprint** **Goal**

The sprint goal is a concise statement that outlines the main objective of a sprint. It helps the team focus on a single, clear outcome rather than scattered tasks. The goal is set collaboratively during sprint planning with the product owner and should be realistic, measurable, and achievable within the sprint timeframe.

- **Sprint Duration**

Sprint duration is the set period assigned to a sprint, usually lasting between 1 and 4 weeks. In this project, each sprint is set to last three weeks, enabling to deliver working increments, collect feedback, and make necessary adjustments to priorities.

- **Product** **Increment**

The product increment is the functional output produced at the end of each sprint, encompassing all features that meet the Definition of Done. Each increment builds on previous ones, gradually forming a complete working product and delivering tangible value with every sprint.

- . (GeeksforGeeks, 2025)

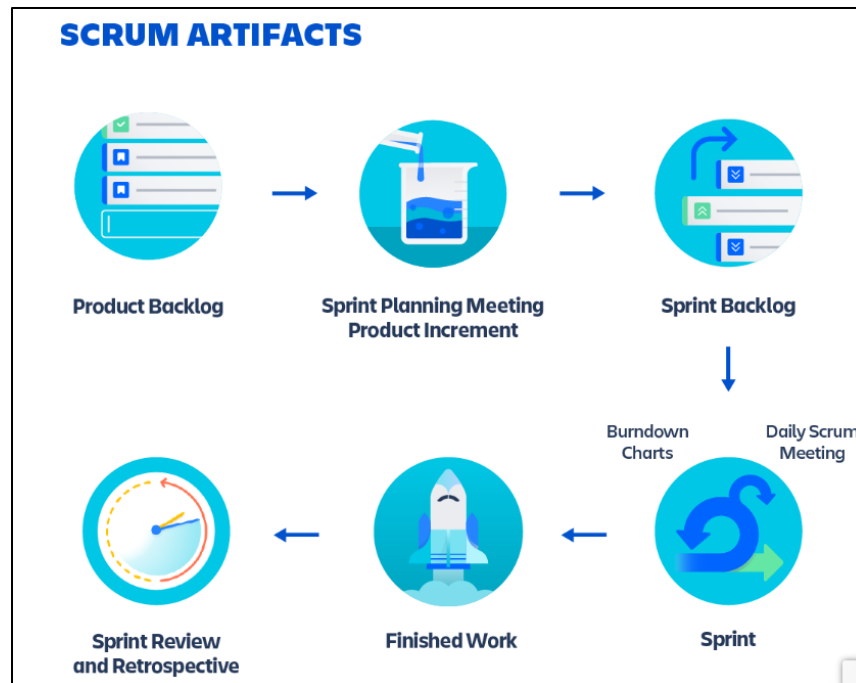


Figure 2: Scrum artifacts (ATLASSIAN, 2024)

For this project, Scrum has been chosen as the main development methodology because it supports continuous improvement, frequent feedback, and the flexibility to modify requirements as the project evolves. This approach is particularly effective for platforms with multiple features and shifting user expectations.

Scrum is well-suited for this system since functionalities such as bidding, payments, messaging, and AI-based estimation can be developed incrementally. It allows for early testing, gathering feedback, and making improvements, delivering functional components in each sprint, minimizing risks, and ensuring that the final product aligns with user needs. In this project, each sprint is set to last three weeks, enabling to deliver working increments, collect feedback, and make necessary adjustments to priorities.

3 Work till date

3.1 Requirement Gathering

Requirement analysis marked the starting point of the project and played a crucial role in shaping the overall system design and development process. This stage

concentrated on clearly defining both functional and non-functional requirements by closely examining the expectations and challenges of freelancers as service providers and SMEs as service consumers within the proposed matchmaking platform.

Data for this phase was gathered through a review of academic and technical resources, informal questionnaires, and an evaluation of existing freelancing platforms. Special focus was placed on issues commonly faced by local SMEs and student freelancers, including cost limitations, trust concerns, lack of market visibility, and difficulties in securing appropriate projects. This approach ensured that the system requirements were grounded in practical needs rather than abstract assumptions.

All identified requirements were systematically reviewed and documented to reduce uncertainty and prevent unnecessary expansion of the project scope.

[Read More.....](#)

3.1.1 SRS

3.1.1.1 Introduction

3.1.1.1.1 Purpose

The proposed system is a web-based platform developed to link Small and Medium Enterprises (SMEs) with freelancers, particularly IT students and professionals at an early stage of their careers. SMEs can publish project details, while freelancers can explore available opportunities, submit proposals, and collaborate online. The platform is intended to offer a secure, cost-effective, and locally focused alternative to existing global freelancing systems.

[Read More....](#)

3.1.2 Product backlog

ID	User Story	Acceptance Criteria	Priority

PB1	As a user, I want to create an account so I can start using the platform.	Correct input fields, phone/email verification, unique account check, KYC option available.	High
PB2	As a user, I want to log in securely so I can access my dashboard.	Password encryption, proper session management, clear error handling.	High
PB3	As a user, I want to update my profile whenever needed.	Editable details, profile image upload, data validation.	Medium
PB4	As a user, I want to upload my KYC documents for verification.	Safe document upload, admin approval required	High
PB5	As an admin, I want to check and verify KYC submissions.	View files, accept or reject with remarks.	High
PB6	As an SME, I want to post a project so freelancers can apply.	Basic project details, skills, budget, file attachments.	High
PB7	As an SME, I want to modify or close my posted project	Editable information and	Medium

		option to archive the project.	
PB8	As a freelancer, I want to browse available projects that match my skills.	Search and filter by skills, budget, or date	High
PB9	As a freelancer, I want to place a bid on projects I'm interested in.	Set bid amount, timeline, and a cover message	High
PB10	As a freelancer, I want to edit or cancel my bid before being hired.	Allowed prior to selection, confirmation message shown.	Medium
PB11	As an SME, I want to view and compare bids to choose the right freelancer.	Access profiles, reviews, and portfolios.	High
PB12	As an SME, I want to officially hire a freelancer through the system.	Status updated, notifications sent.	High
PB13	As an SME, I want to deposit the project amount in escrow for security.	Integrated eSewa payment, confirmation receipt.	High

PB14	As a freelancer, I want assurance that payment is reserved in escrow.	Visible escrow status and release terms.	High
PB15	As an SME, I want to release the payment once the work is done.	Manual or automatic release, record kept in logs.	High
PB16	As a user, I want to chat with my project partner for smooth communication.	Real-time messaging with timestamps	High
PB17	As a user, I want to share files safely during the project	File size limit, basic security checks.	Medium
PB18	As a freelancer, I want an AI tool that gives rough cost and time suggestions	Basic machine-learning/rule-based outputs that can be adjusted.	Medium
PB19	As an admin, I want a dashboard to supervise users, payments, and disputes.	Visual stats, tables, role controls.	High
PB20	As a user, I want to submit disputes if problems arise.	Select issue type, upload evidence	High

PB21	As an admin, I want to review complaints and settle disputes fairly.	Accept/reject decisions with justification.	High
PB22	As an SME, I want to rate freelancers after a project is completed.	Rating scale plus written feedback.	Medium
PB23	As a freelancer, I want to see my overall ratings.	Summary shown on profile.	Medium
PB24	As a user, I want to receive alerts about bids, payments, chats, and disputes.	Real-time or email alerts.	Medium

Table 1: Product Backlog

3.1.3 Pre survey

- Response

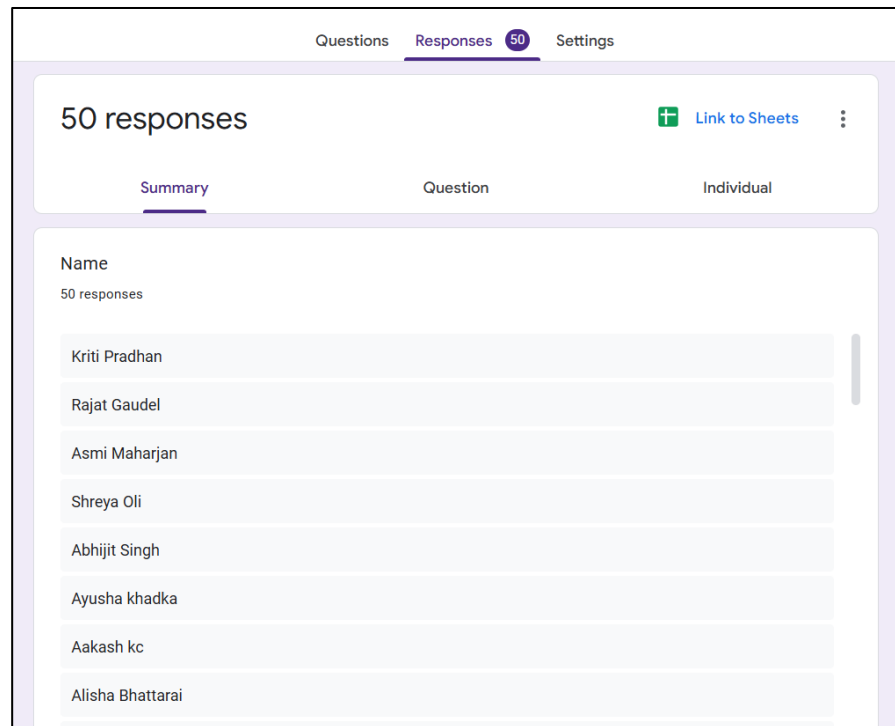


Figure 3:Survey response

- Survey Result

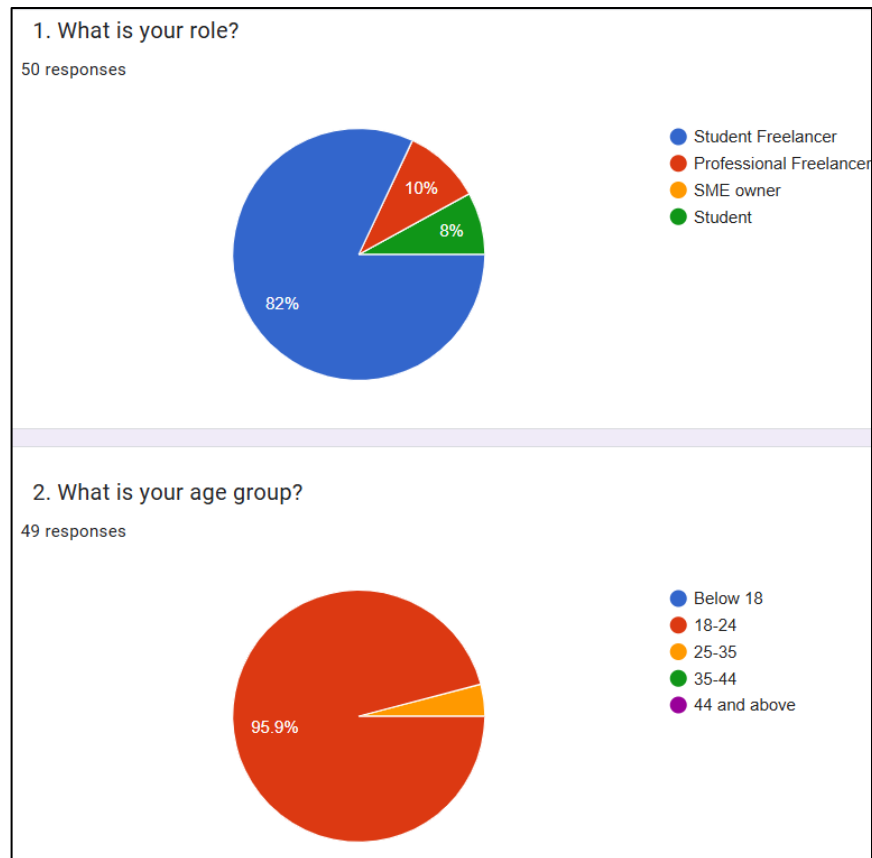


Figure 4: Pre survey question 1 and 2 response

Result: Most respondents identified as Student Freelancers (82%), followed by professional Freelancers (10%) and SME owners (8%), showing that the platform mainly attracts students engaging in freelance work. The vast majority of participants (95.9%) were between 18-24 years old, with a small fraction in the 25-35 age range, indicating a predominantly young adult user base.

[See More....](#)

3.2 System Design

3.2.1 Use case

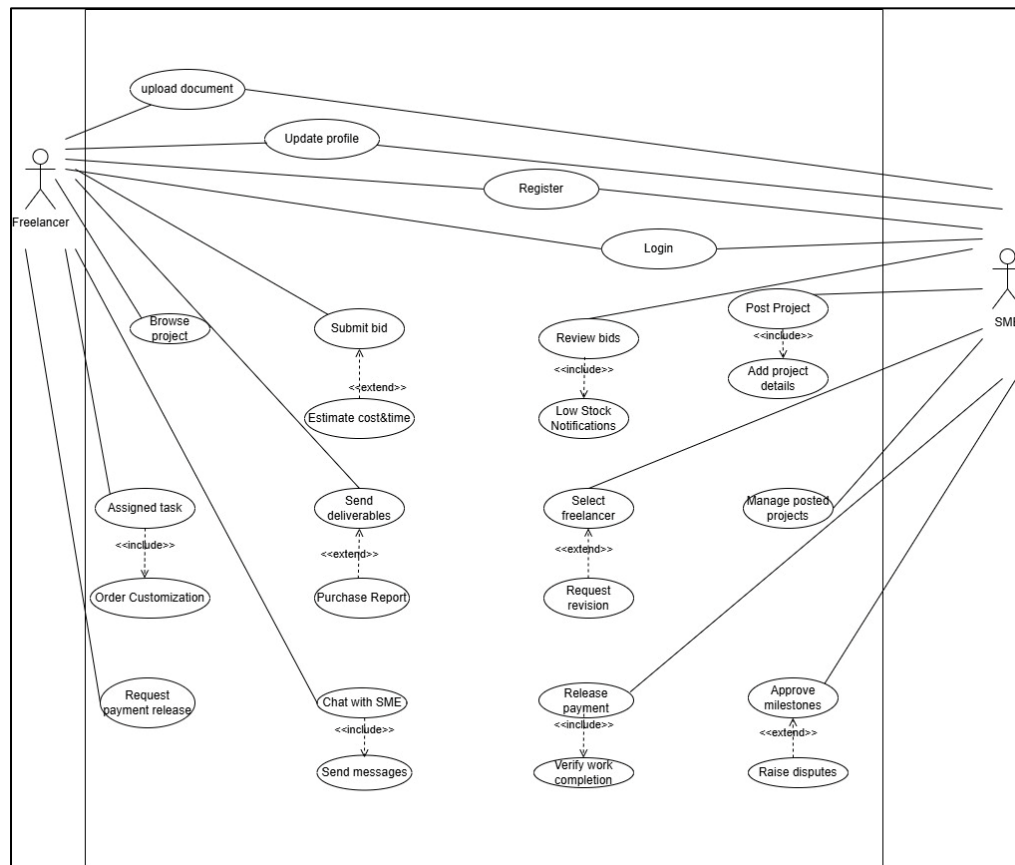


Figure 5: Use case diagram

3.2.2 Entity Relationship Diagram

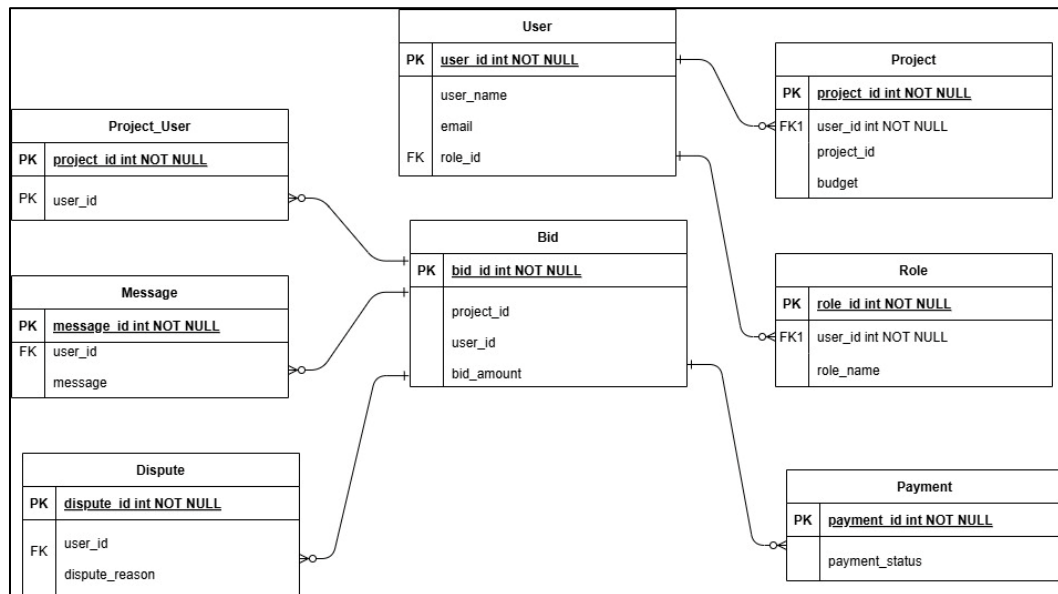


Figure 6: Erd diagram

3.2.3 Class Diagram

A UML class diagram illustrates how a system is organized by depicting its classes along with their attributes, operations, and the connections that exist between them. (GeeksforGeeks, 2025)

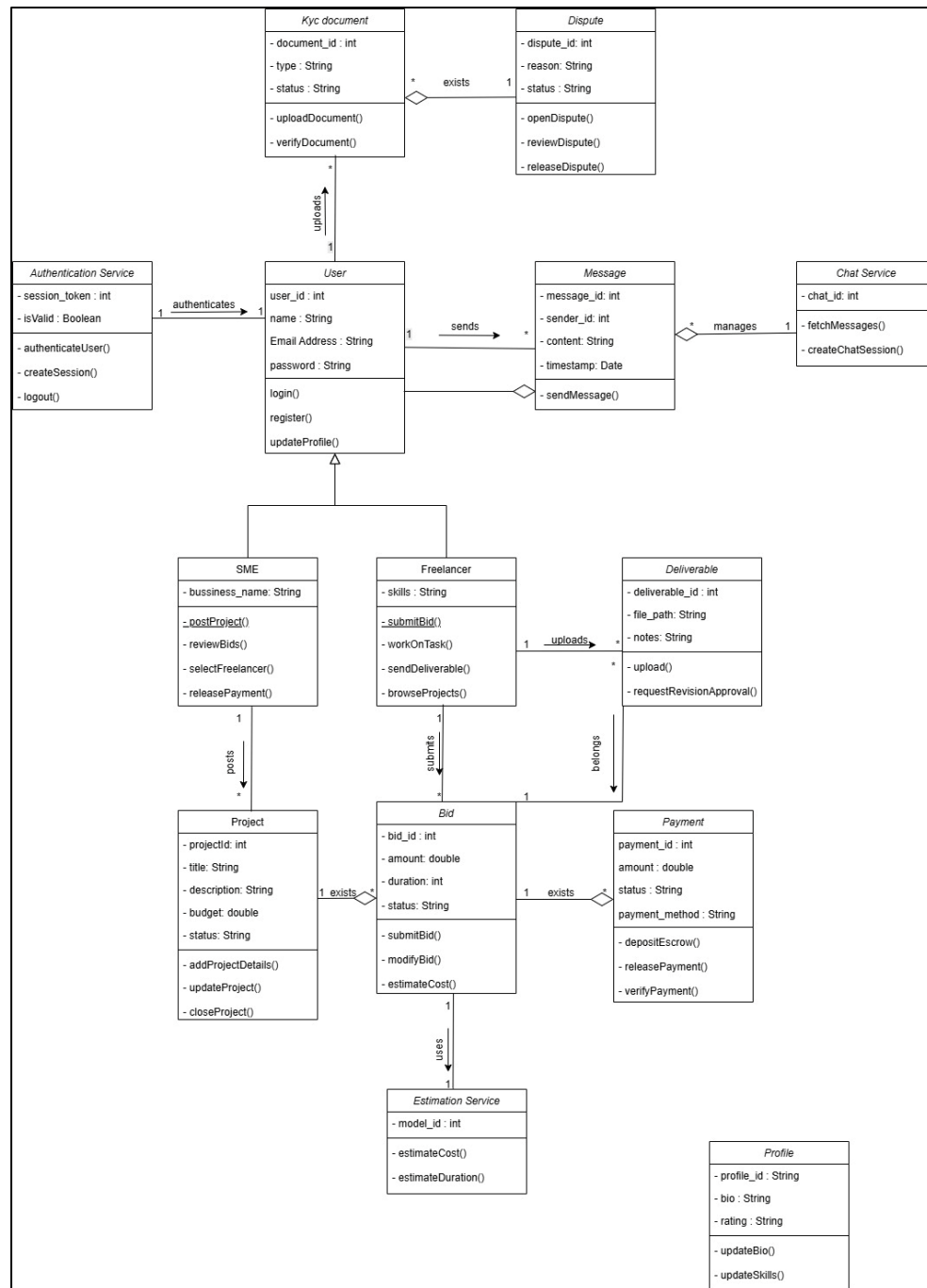


Figure 7: Class diagram

3.2.4 Data Flow Diagram

A data flow diagram (DFD) shows how data moves within an information system or business process, using a visual format to represent inputs, processing steps, storage, and outputs. (IBM, 2025)

3.2.4.1 Context diagram

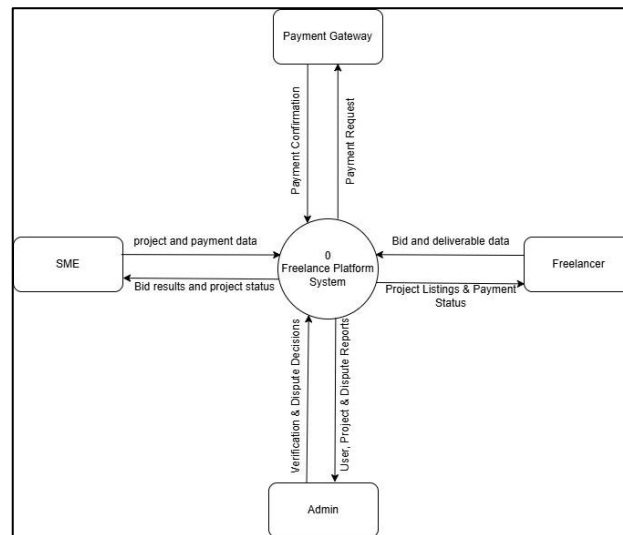


Figure 8: Context level diagram

3.2.4.2 DFD 1

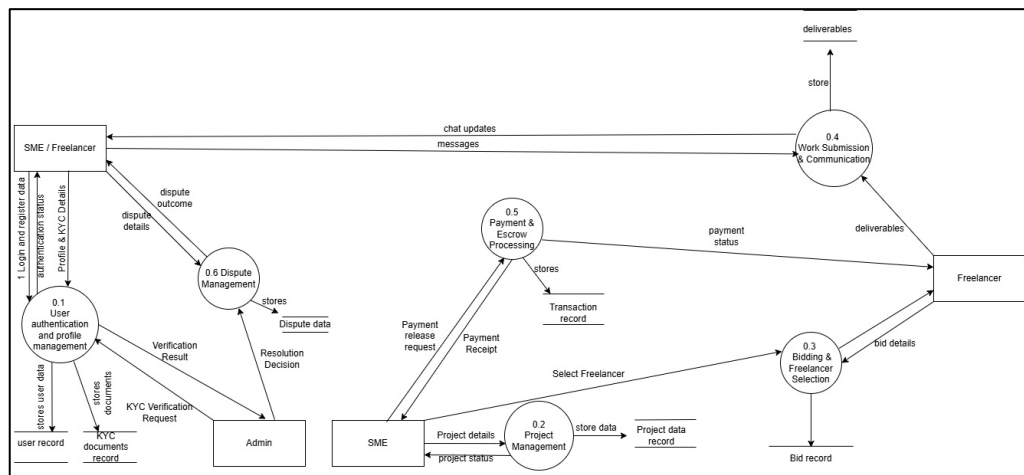


Figure 9: DFD 1

3.2.4.3 DFD 2

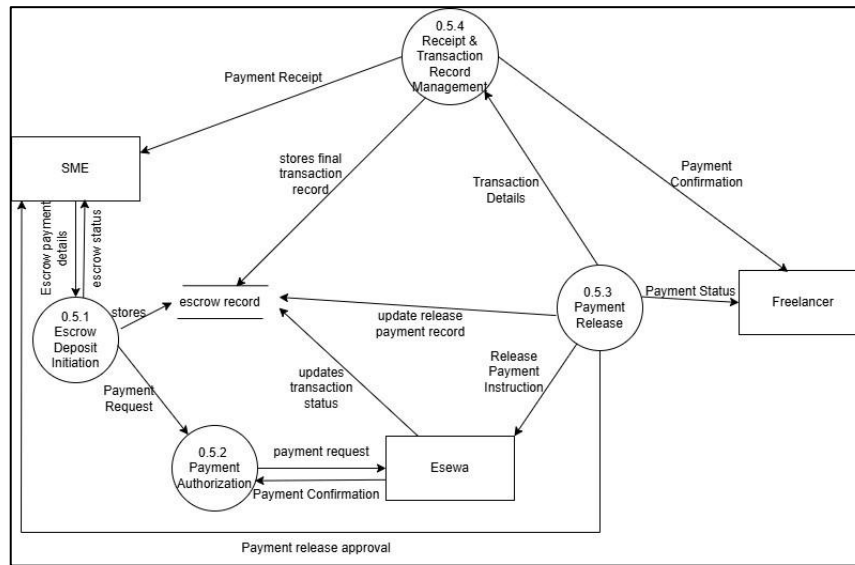


Figure 10: DFD 2

3.3 Sprint 1 / Prototype 1 – Login and Register

3.3.1 Sprint Planning

[Read More....](#)

3.3.2 Design

[Read More....](#)

3.3.2.1 High Level Use case

[Read More....](#)

3.3.2.2 Expanded Use case

[Read More....](#)

3.3.2.3 Activity Diagram

In UML, an activity diagram depicts a system's behavior by outlining the order of actions within a process. (IBM, 2025)

- Login

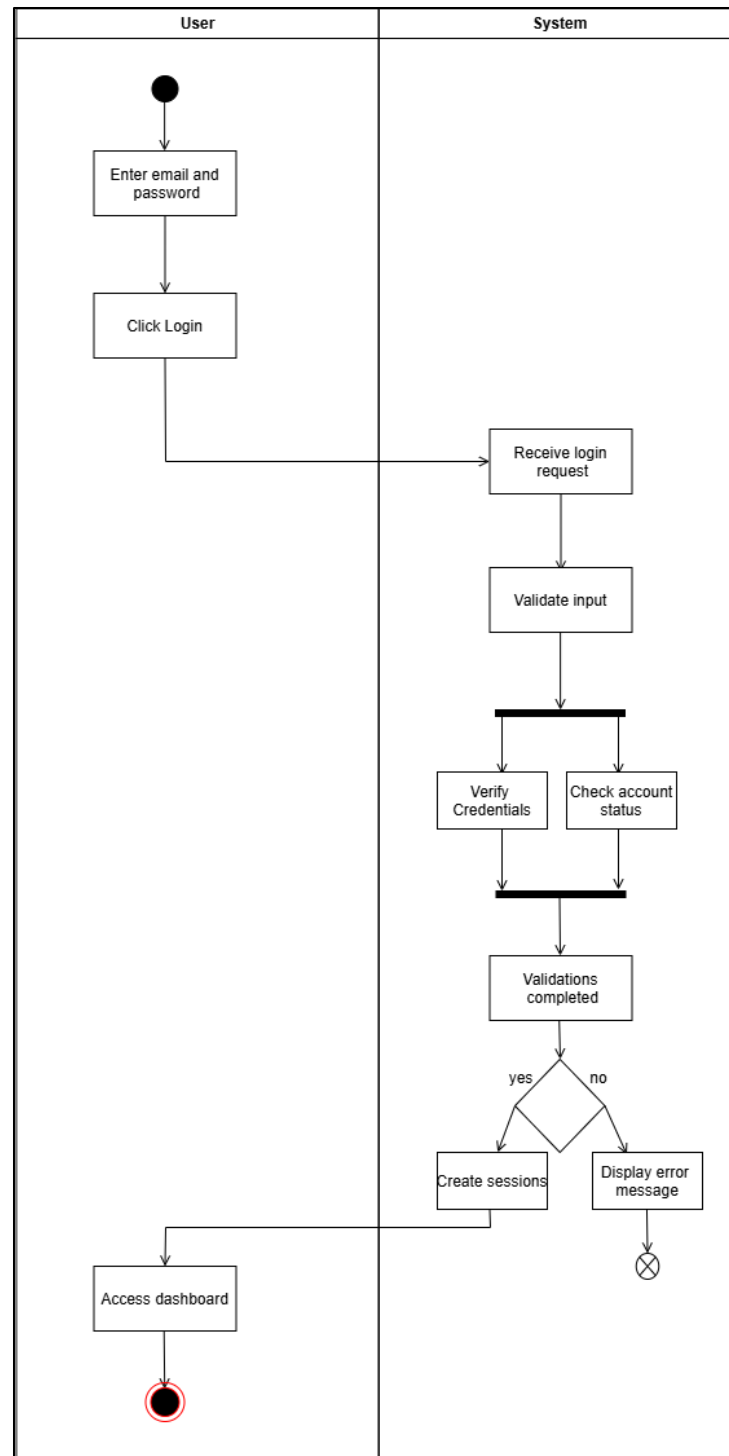


Figure 11: Login activity diagram

- Registration

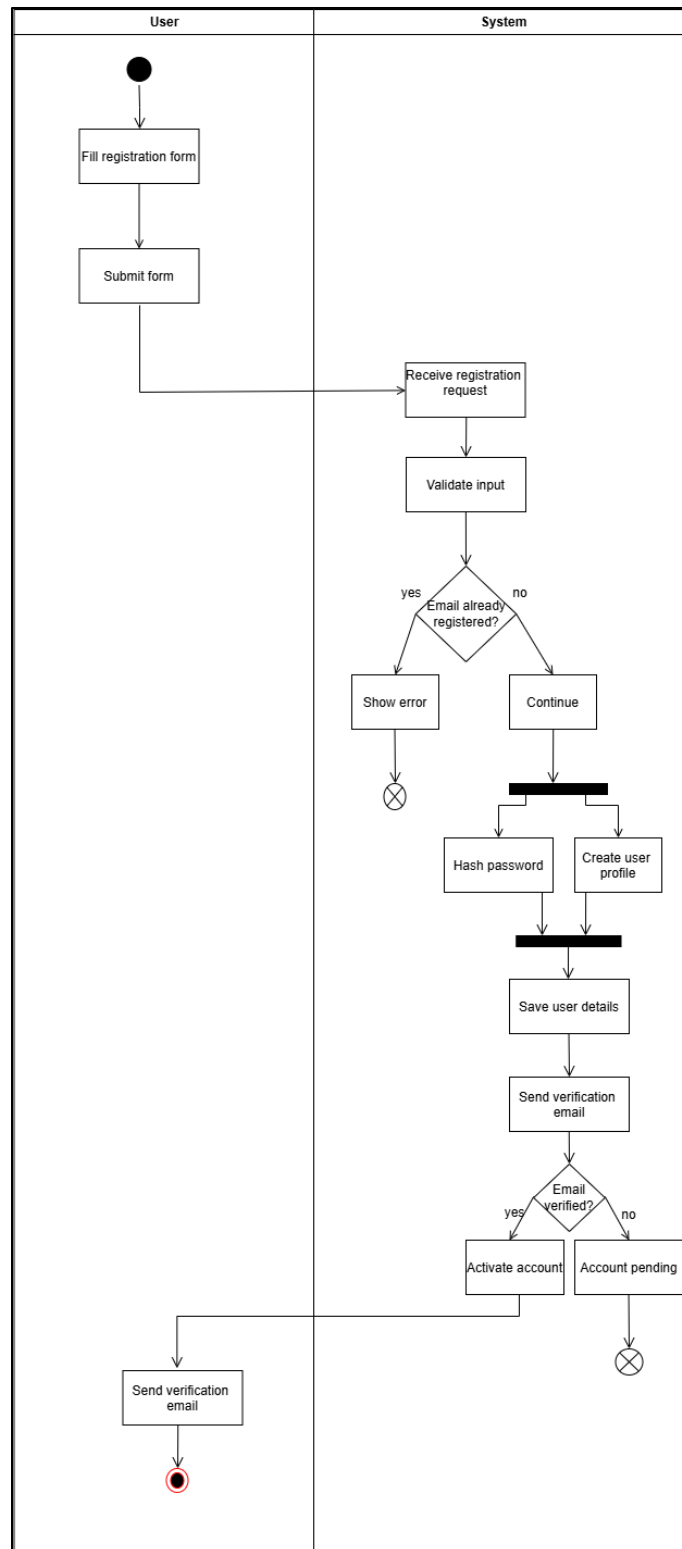


Figure 12: Register activity diagram

3.3.2.4 Sequence Diagram

A Sequence Diagram illustrates how objects interact with each other in a specific sequence. (GeeksforGeeks, 2025)

- Registration

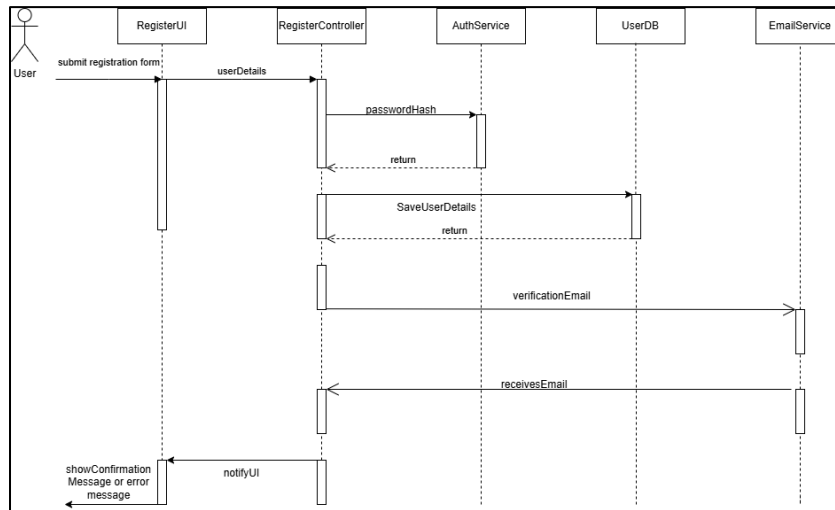


Figure 13: Registration sequence diagram

- Login

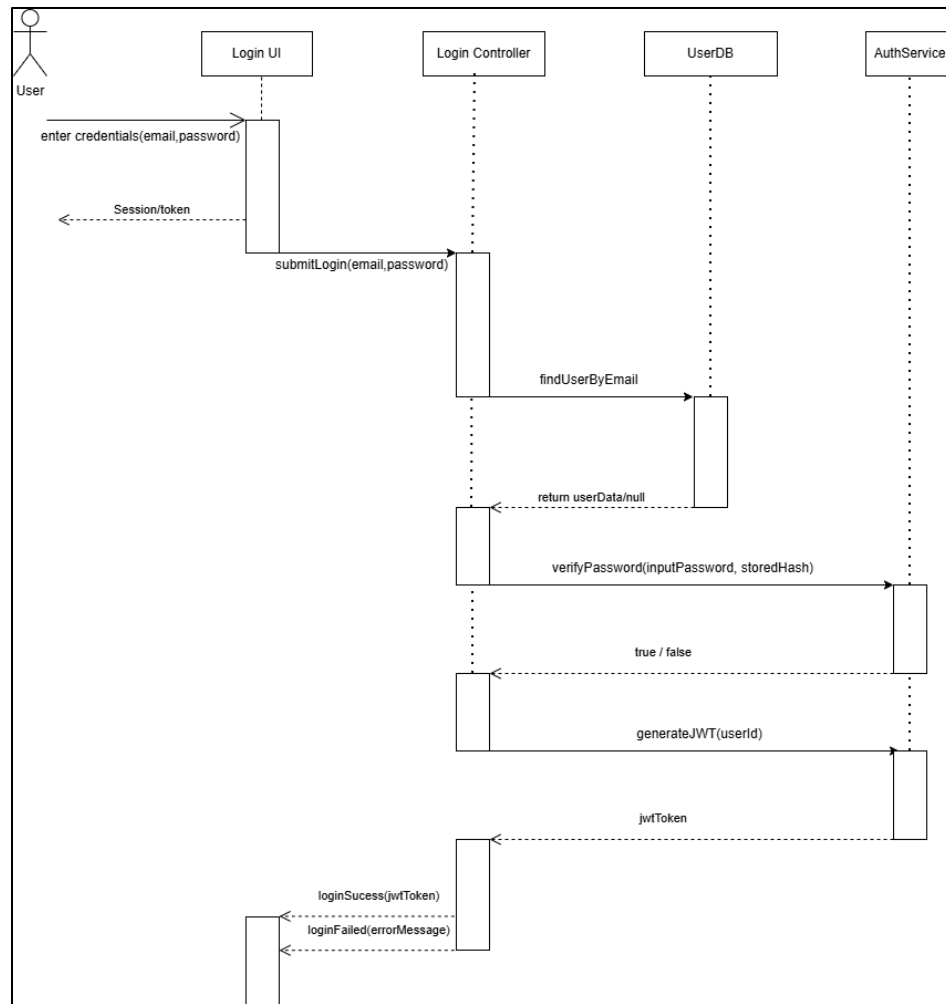


Figure 14: Login sequence diagram

3.3.2.5 Wireframe

- Login



The diagram shows a login form wireframe. It is enclosed in a large rectangular border. Inside, there is a central container with a title "Sign in to your account" in a large, bold font. Below the title is a subtitle "Welcome back. Please enter your details". There are three input fields: the first is labeled "Email address", the second is labeled "Password", and the third is a button labeled "Sign In". Below the button is a link "Don't have an account? Sign Up".

Sign in to your account

Welcome back. Please enter your details

Email address

Password

Sign In

Don't have an account? Sign Up

Figure 15: Login wireframe

- Register

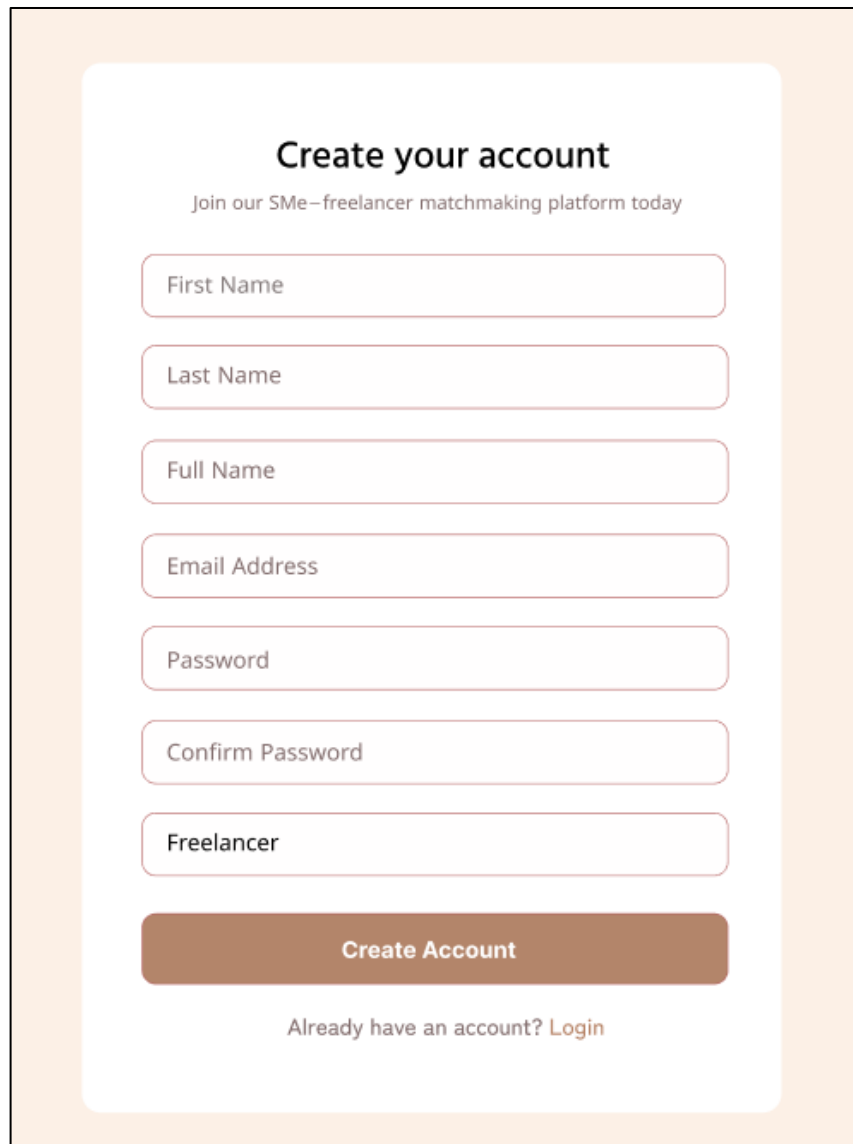
The wireframe shows a registration form with the following elements:

- Title:** "Create your account" in a large, bold font.
- Subtitle:** "Join our SME-freelancer matchmaking platform today" in a smaller font.
- Input Fields:** Seven vertical text input fields with placeholder text: "First Name", "Last Name", "Full Name", "Email Address", "Password", "Confirm Password", and "Freelancer". The "Freelancer" field includes a dropdown arrow icon on the right.
- Submit Button:** A wide button labeled "Create Account" centered below the input fields.
- Footer Link:** A link "Already have an account? Login" centered at the bottom of the form.

Figure 16: Register Wireframe

3.3.2.6 Mockup

- Registration



The image shows a registration form titled "Create your account" with a subtitle "Join our SME-freelancer matchmaking platform today". The form is set against a light orange background. It contains seven input fields: "First Name", "Last Name", "Full Name", "Email Address", "Password", "Confirm Password", and "Freelancer". Below these fields is a brown "Create Account" button. At the bottom, there is a link that says "Already have an account? Login".

Create your account

Join our SME-freelancer matchmaking platform today

First Name

Last Name

Full Name

Email Address

Password

Confirm Password

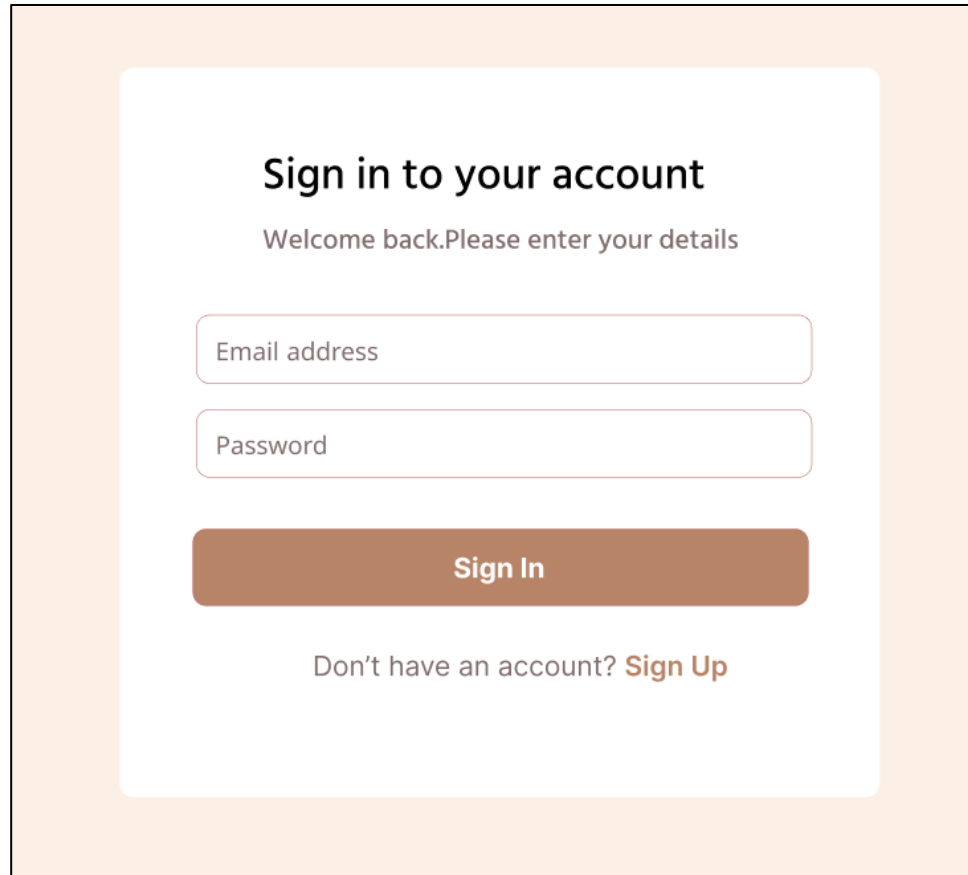
Freelancer

Create Account

Already have an account? [Login](#)

Figure 17: Registration UI mockup

- Login

A login form mockup with a light orange background. The form is centered and contains the following elements: a title "Sign in to your account", a subtitle "Welcome back. Please enter your details", two input fields labeled "Email address" and "Password", a "Sign In" button, and a link "Don't have an account? Sign Up".

Sign in to your account

Welcome back. Please enter your details

Email address

Password

Sign In

Don't have an account? [Sign Up](#)

Figure 18: Login Mockup

3.3.3 Development

3.3.3.1 Frontend

3.3.3.1.1 Login

- Login UI

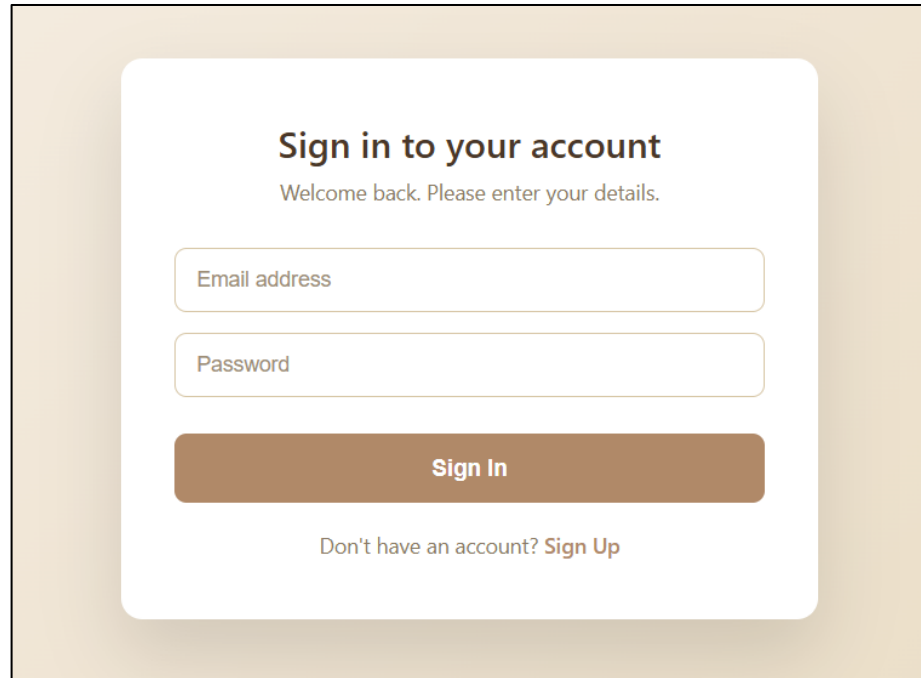
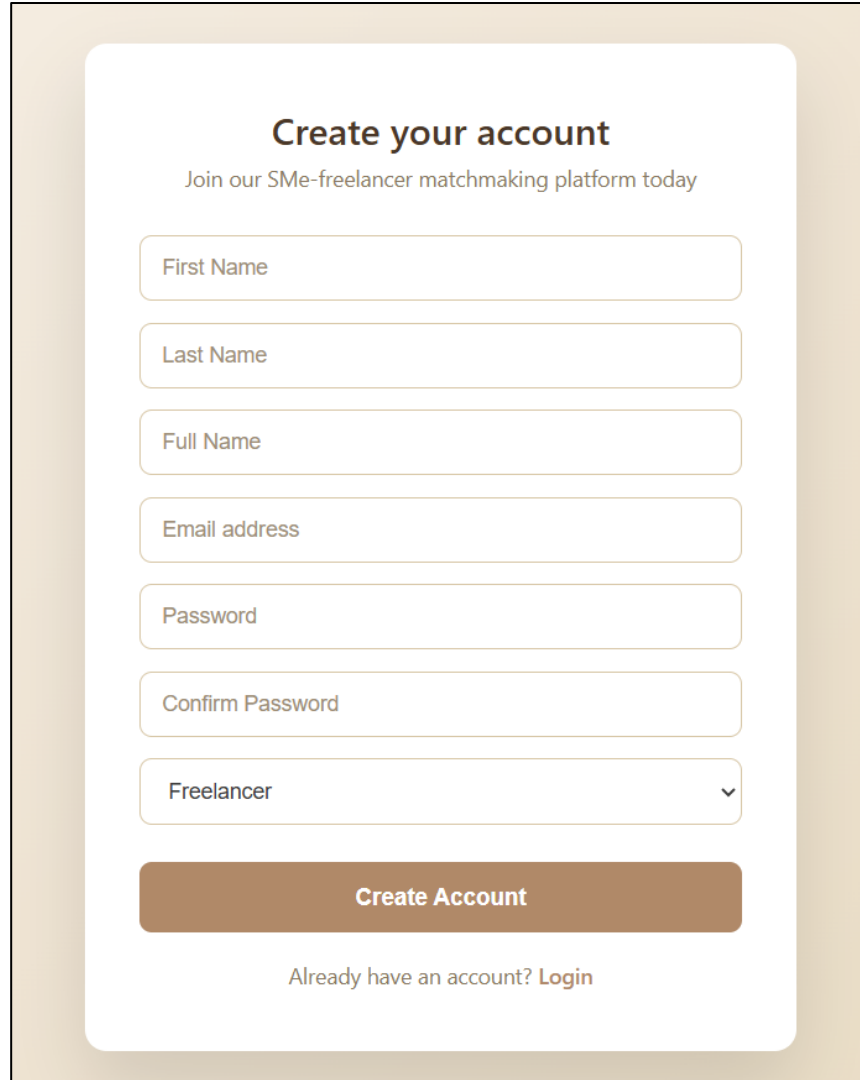
A login form UI design. It features a light beige background with a white rounded rectangle in the center. Inside the white rectangle, the text "Sign in to your account" is displayed in a bold, dark brown font. Below it, a smaller line of text reads "Welcome back. Please enter your details." There are two input fields: "Email address" and "Password", both with light beige borders. Below these fields is a solid brown "Sign In" button. At the bottom of the white rectangle, the text "Don't have an account? Sign Up" is shown, with "Sign Up" as a link.

Figure 19: Login UI

- Register UI

A registration form titled "Create your account" with a subtitle "Join our SME-freelancer matchmaking platform today". The form contains seven input fields: "First Name", "Last Name", "Full Name", "Email address", "Password", "Confirm Password", and a dropdown menu labeled "Freelancer" with a downward arrow. Below the fields is a brown "Create Account" button. At the bottom, there is a link "Already have an account? Login".

Create your account

Join our SME-freelancer matchmaking platform today

First Name

Last Name

Full Name

Email address

Password

Confirm Password

Freelancer ▼

Create Account

Already have an account? [Login](#)

Figure 20: Register UI

[See More....](#)

3.3.3.2 Backend

[See More....](#)

3.3.4 Sprint Review

[Read More....](#)

3.3.5 Sprint retrospective

[Read More...](#)

3.3.6 Velocity Chart

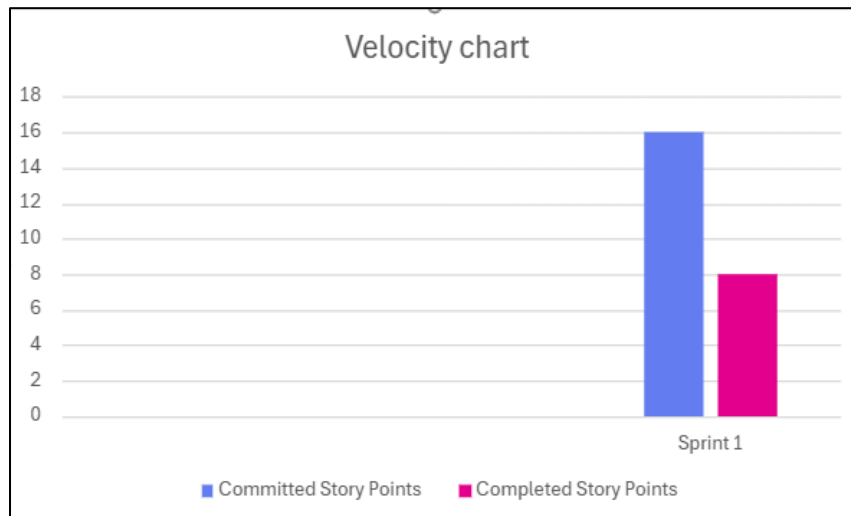


Figure 21: Velocity chart

4 Progress Analysis

4.1 Gantt chart

A Gantt chart is a timeline that shows project tasks and milestones in a visual format.
(ATLASSIAN, 2025)

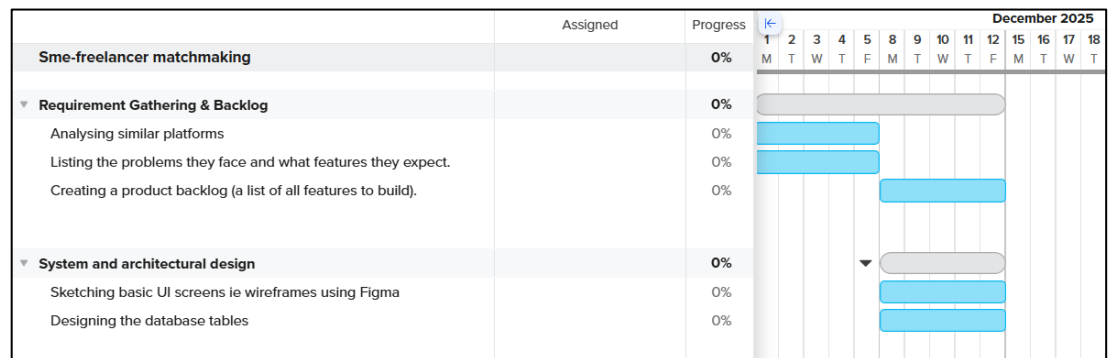


Figure 22: Gantt chart

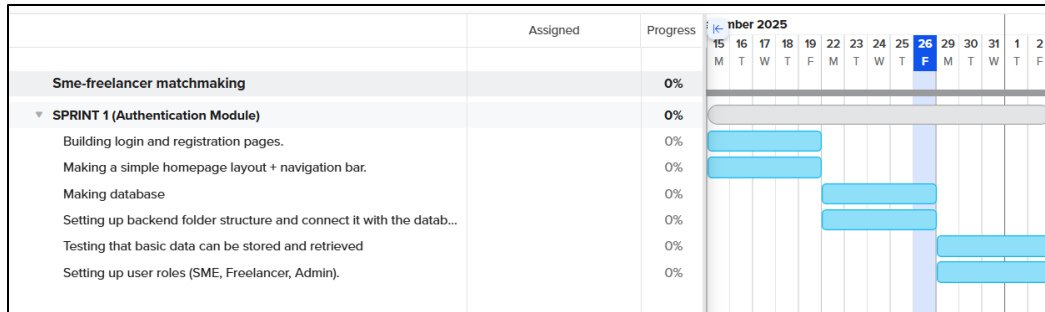


Figure 23: Gantt chart sprint 1

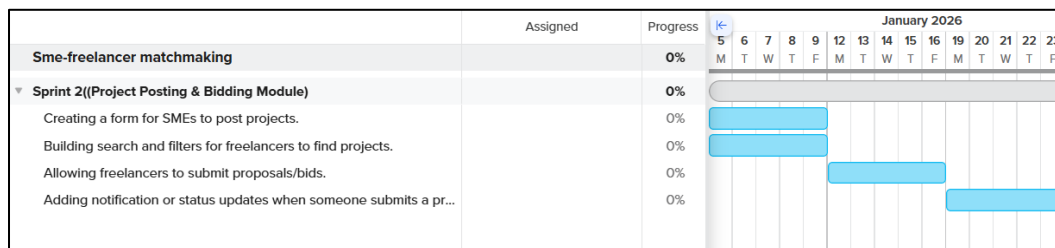


Figure 24: Gantt chart sprint 2

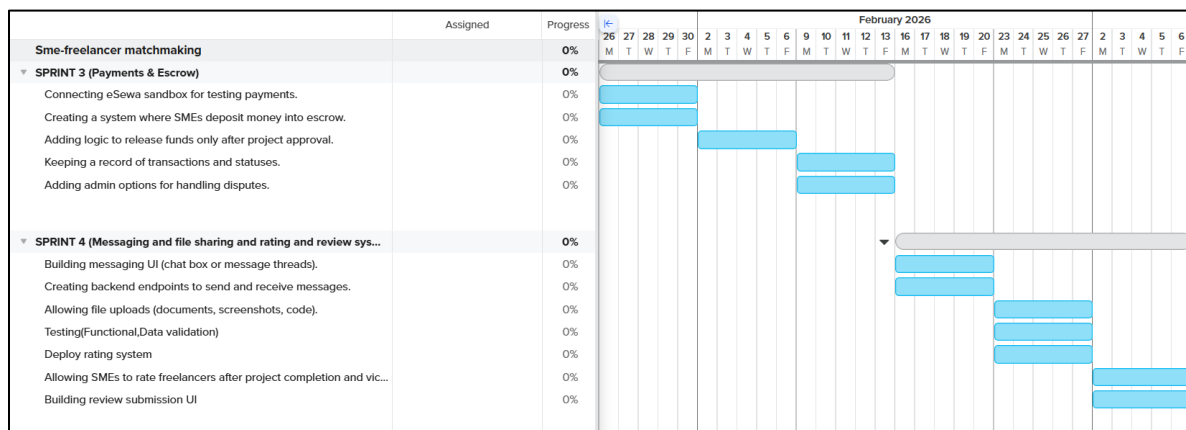


Figure 25: Gantt chart sprint 3 and 4

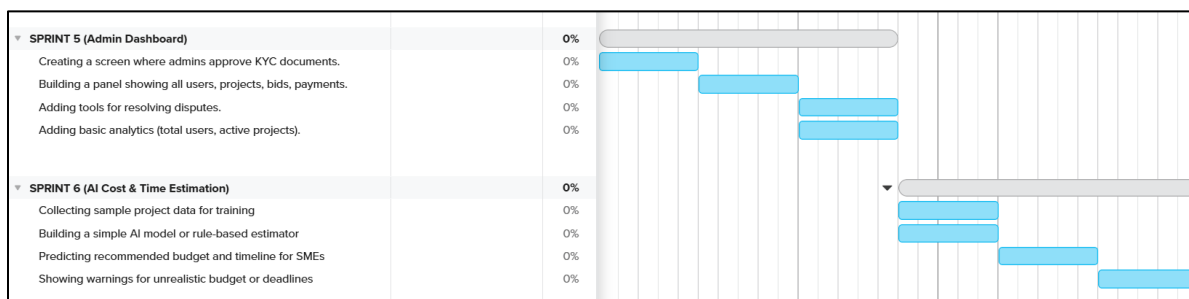


Figure 26: Gantt chart sprint 5 and 6

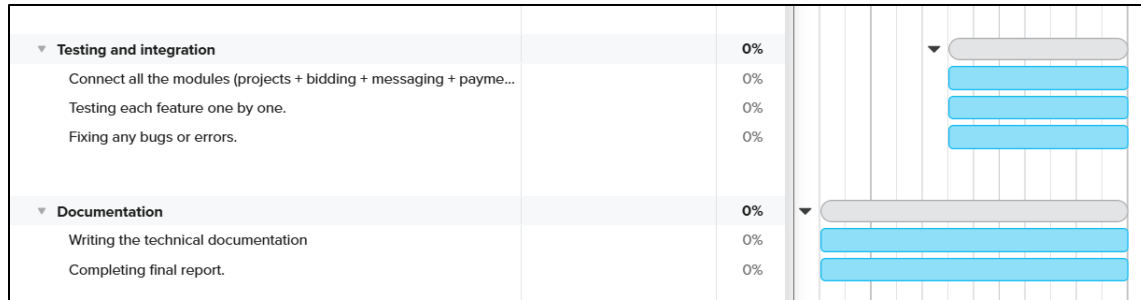


Figure 27: Testing and Documentation gantt chart

4.2 Progress Table

Activity	Planned Duration	Status	Completion
Problem identification and analysis	1 week	Completed	100%
Requirement Gathering	1 week	Completed	100%
SRS Documentation	1 week	Completed	100%
Pre-Survey and Research	1 week	Completed	100%
Use Case Diagram & Descriptions	1 week	Completed	100%
ER Diagram	1 week	Completed	100%
Class Diagram	1 week	Completed	100%
Sprint 1 Planning	3 days	Completed	100%
Login & Registration Design (UML)	1 week	Completed	100%
Wireframe & Mockup Design	1 week	Almost Completed	80%
Frontend Development (Sprint 1)	3 weeks	Completed	100%

Backend Development (Sprint 1)	3 weeks	Completed	90%
Authentication Implementation	1 week	Completed	90%
Sprint Review & Retrospective	3 days	Not completed	0%
Sprint 2 Planning	3 days	Not started	0%

Table 2: Progress table

4.3 Analysis

An evaluation of the project timeline and progress records shows that development is moving forward in a structured and well-managed manner. Initial phases such as problem analysis, requirement collection, SRS preparation, and preliminary research were completed within the planned timeframe. Completing these activities early helped establish a clear understanding of system goals and reduced uncertainties during later stages of development.

The design phase was carried out successfully, covering system architecture, UML diagrams, data flow representations, and interface mockups. These deliverables were finalized on schedule, allowing development work to proceed smoothly. Although small design adjustments were required during this phase, they were addressed efficiently and did not cause noticeable delays in the project schedule.

[Read More....](#)

5 Further works

5.1 Sprint 2: Project Posting and Bidding Module

[Read More....](#)

5.2 Sprint 3: Escrow and Payment Management

[Read More....](#)

5.3 Sprint 4: Communication, File Sharing, and Review System

[Read More....](#)

5.4 Sprint 5: Administrative Dashboard and KYC Verification

[Read More....](#)

5.5 Sprint 6: AI-Based Cost and Timeline Estimation

[Read More....](#)

5.6 Sprint 7: System Integration, Testing, and Optimization

[Read More....](#)

6 Conclusion

This project aimed to develop a platform that connects student and early-career IT freelancers with Small and Medium Enterprises (SMEs) in Nepal. SMEs often face challenges in accessing affordable and reliable IT services, while students struggle to find real-world projects to build experience and portfolios. The proposed system bridges this gap by providing a secure, transparent, and structured environment for collaboration, ensuring benefits for both parties.

The project followed a methodical approach, beginning with requirement analysis, system modeling, and architecture design. Core features like secure login, role-based access, project posting, bidding, and communication were planned and partially implemented using the Agile Scrum methodology. Using Scrum allowed iterative progress, regular feedback, and careful tracking of each development phase, ensuring that the system evolves steadily while accommodating adjustments.

The work completed so far shows consistent and structured progress. Deliverables such as the Software Requirements Specification, UML diagrams, database design, and the first development sprint were completed successfully. Sprint 1 focused on authentication and access control, establishing a strong foundation for the remaining modules. The progress analysis indicates that the project is on track and prepared for the next stages of development.

Upcoming sprints will add more functionality, including project posting and bidding, escrow-based payments, messaging and file sharing, administrative tools, and AI-powered cost and timeline predictions. These features will enhance transparency, trust, and planning for both SMEs and freelancers. By integrating intelligent estimation tools and robust administrative oversight, the system is designed to offer a locally relevant alternative to global freelance platforms.

In summary, the Freelancer–SME Matchmaking Platform has the potential to be an effective, practical, and impactful solution. It meets academic objectives by applying real-world software engineering practices, while also promoting digital adoption, skill development, and professional opportunities within Nepal's SME sector. With continued development, the platform can become a fully functional tool that benefits both freelancers and SMEs in a sustainable way.

7 References

ATLASSIAN, 2024. *ATLASSIAN*. [Online]

Available at: <https://www.atlassian.com/agile/scrum>

[Accessed 30 11 2025].

ATLASSIAN, 2025. *ATLASSIAN*. [Online]

Available at: <https://www.atlassian.com/agile/software-development/sdlc>

[Accessed 3 December 2025].

ATLASSIAN, 2025. *ATLASSIAN*. [Online]

Available at: <https://www.atlassian.com/agile/project-management/gantt-chart>

[Accessed 3 December 2025].

FNCCI, 2024. *FNCCI*. [Online]

Available at:

https://fncci.org/uploads/publication/file/FNCCI%20Journal%202024_20240710050833.pdf

[Accessed 30 November 2025].

FRIEDRICH NAUMANN FOUNDATION, 2024. *FRIEDRICH NAUMANN FOUNDATION*. [Online]

Available at: <https://www.freiheit.org/south-asia/critical-role-smes-nepals-economic-development>

[Accessed 30 November 2025].

GeeksforGeeks, 2025. *GeeksforGeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering/scrum-artifacts-definitions-and-examples/>

[Accessed 30 November 2025].

GeeksforGeeks, 2025. *GeeksforGeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/>

[Accessed 30 December 2025].

GeeksforGeeks, 2025. *GeeksforGeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-sequence-diagrams/>

[Accessed 30 December 2025].

guru, 2025. *guru*. [Online]

Available at: <https://www.guru.com/why-guru/>

[Accessed 20 December 2025].

IBM, 2025. *IBM*. [Online]

Available at: <https://www.ibm.com/think/topics/data-flow-diagram>

[Accessed 30 December 2025].

IBM, 2025. *IBM*. [Online]

Available at: <https://www.ibm.com/docs/en/rational-soft-arch/9.6.1?topic=diagrams-activity>

[Accessed 30 December 2025].

LinkedIn, 2025. *LinkedIn*. [Online]

Available at: <https://www.linkedin.com/company/upwork/about/>

[Accessed 20 December 2025].

Medium, 2025. *Medium*. [Online]

Available at: <https://medium.com/freelancers-hub/fiverr-review-2025-for-business-owners-and-freelancers-7c5c02fc6bc4>

[Accessed 20 December 2025].

The Himalayan, 2025. *The Himalayan*. [Online]

Available at: <https://thehimalayantimes.com/opinion/beyond-the-gadget-fixer-rethinking-it-in-nepal>

[Accessed 1 December 2025].

Toptal, 2025. *Toptal*. [Online]

Available at: <https://www.toptal.com/>

[Accessed 20 December 2025].

8 Appendix

8.1 Problem Scenario

8.1.1 Local Scenario

In Nepal, small and medium enterprises make up a substantial portion of registered businesses and play an important role in creating jobs and supporting economic stability. Even so, many of these businesses rely on very limited technology, mainly because of tight budgets, complicated processes, and difficulty in finding dependable IT support. For smaller firms, adopting digital solutions is often seen as expensive and risky rather than beneficial.

At the same time, the number of IT graduates in Nepal continues to rise every year. However, a large portion of these graduates struggle to find suitable employment due to a lack of practical experience and professional connections. The absence of dedicated platforms that connect local freelancers with businesses further

deepens this gap, pointing to the need for a solution that aligns with Nepal's specific economic and technological context.

8.1.2 Global Scenario

Globally, online freelancing platforms have transformed the way organizations obtain technical skills and professional services. Small and medium enterprises across different countries now depend on digital marketplaces to access cost-effective solutions, while freelancers enjoy greater flexibility in how and where they work. The rise of remote work worldwide has further strengthened the importance of these platforms.

Despite this growth, many international freelancing platforms remain difficult for student freelancers from developing nations to enter. Strong worldwide competition, payment system limitations, and the lack of guidance for newcomers often restrict their chances of success. These challenges highlight the need for locally focused platforms that incorporate international standards while remaining sensitive to regional conditions and needs.

8.2 Background

8.2.1 About the targeted users

On the other hand, freelancers, especially students and newly graduated IT professionals, generally have strong academic knowledge and technical abilities but encounter difficulties when entering the professional workforce. Factors such as limited exposure to real projects, absence of professional connections, and minimal industry experience restrict their access to suitable freelance work. The proposed system seeks to overcome these challenges by providing a structured digital platform that connects SMEs with skilled freelancers, enabling productive collaboration that benefits both parties.

8.2.2 Understanding the solution

The system includes key functionalities such as secure user login, role-based access, project tracking features, and built-in communication tools to support clear and efficient interaction. By organizing the entire collaboration process within a

single platform, it helps improve transparency, reduce delays, and avoid common misunderstandings. Furthermore, the solution encourages local digital engagement while ensuring strong data protection, user-friendly design, and dependable system performance.

8.3 Review of similar projects

8.3.1 Project 1- Upwork

Upwork is a well-known international freelancing platform that links organizations with skilled professionals from different disciplines worldwide. The platform supports features such as project advertisements, bidding systems, direct communication, and built-in payment handling. Despite offering a wide range of opportunities, the marketplace is extremely competitive, which often makes it challenging for new freelancers to stand out and obtain projects. Moreover, comparatively high platform fees and strong global competition can restrict access for students and small local businesses. Upwork is a global online marketplace that links organizations with independent professionals from around the world. It supports a wide range of clients, from individual entrepreneurs to large multinational corporations, by offering a reliable and secure platform for collaboration. Through Upwork, businesses and freelancers can work together in flexible and innovative ways that help both sides grow. The platform hosts a diverse talent pool with expertise in thousands of skill areas, including web and application development, design and creativity, customer service, finance and accounting, consulting, and business operations. (LinkedIn, 2025)

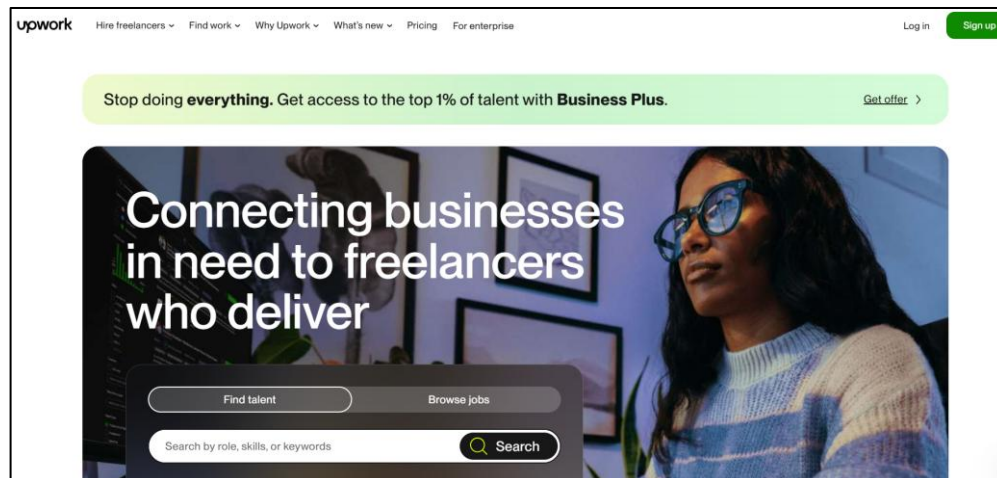


Figure 28: Upwork

8.3.2 Project 2 – Fiverr

Fiverr is a well-known online marketplace where freelancers can offer or purchase digital services at reasonable prices, including design, writing, tech support, and marketing. In running my online writing business for four years, I've relied on Fiverr to hire ghostwriters, create AI-generated headshots, edit images, and more, always achieving high-quality results. The platform has supported me in building a strong personal brand, streamlining my workflow, and expanding my business. With more than 600 service categories and 3.8 million active buyers, Fiverr provides ample opportunities to find skilled talent and connect with clients. Its intuitive interface, cost-effective services, and access to top freelancers make it ideal for outsourcing work. To get started, visit Fiverr, search for the service you need, and carefully evaluate freelancer profiles. (Medium, 2025)

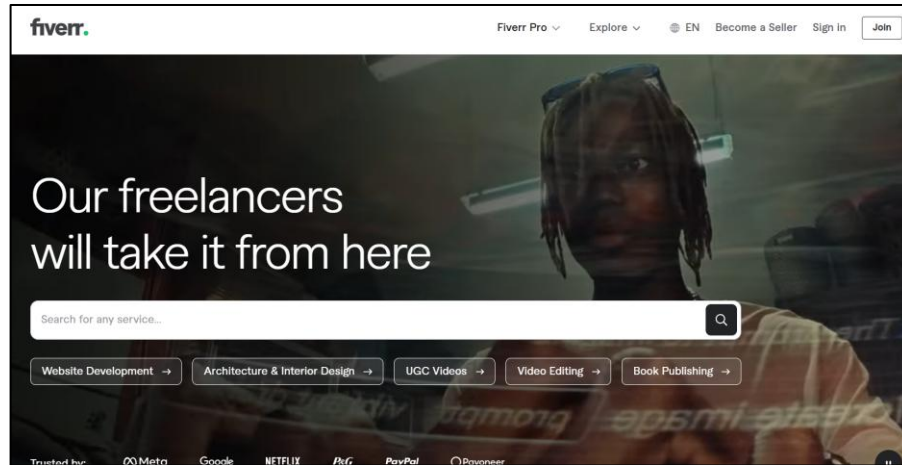


Figure 29: Fiverr

8.3.3 Project 3 – Freelancer.com

Freelancer.com is a widely used digital marketplace that enables freelancers to bid on projects and connect with clients globally. It covers a broad range of industries and project types. However, freelancers often face difficulties due to intense competition, service fees, and payment-related constraints. The absence of structured guidance and localized support further reduces its effectiveness for student freelancers and beginners, particularly in developing regions.

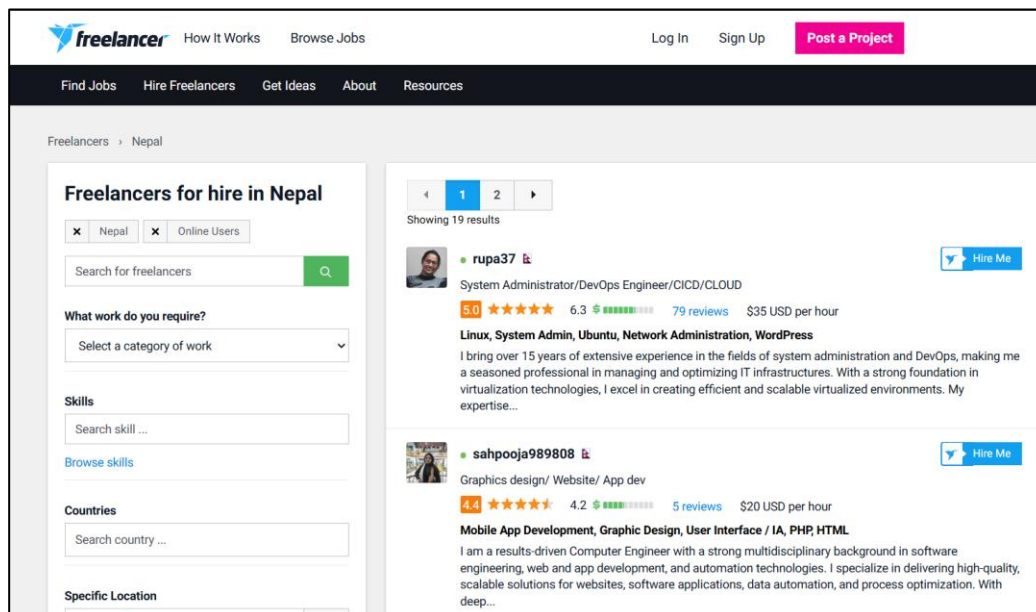


Figure 30: Freelancer.com

8.3.4 Project 4 – Toptal

Toptal is a premier, highly selective professional network that connects top-tier talent from around the world in fields such as software development, design, marketing, consulting, product management, and project leadership. The platform maintains a high standard by admitting only the most skilled and experienced professionals, ensuring organizations can rely on its talent pool. From innovative startups to large multinational corporations, companies turn to Toptal to access exceptional experts for critical, high-priority, and technically challenging projects. By offering a curated selection of elite professionals, Toptal enables businesses to carry out complex initiatives with confidence, lower hiring risks, and achieve project goals efficiently. The platform also focuses on rigorous screening, ongoing support, and smooth collaboration, guaranteeing that both clients and professionals benefit from a reliable and effective engagement model. (Toptal, 2025)

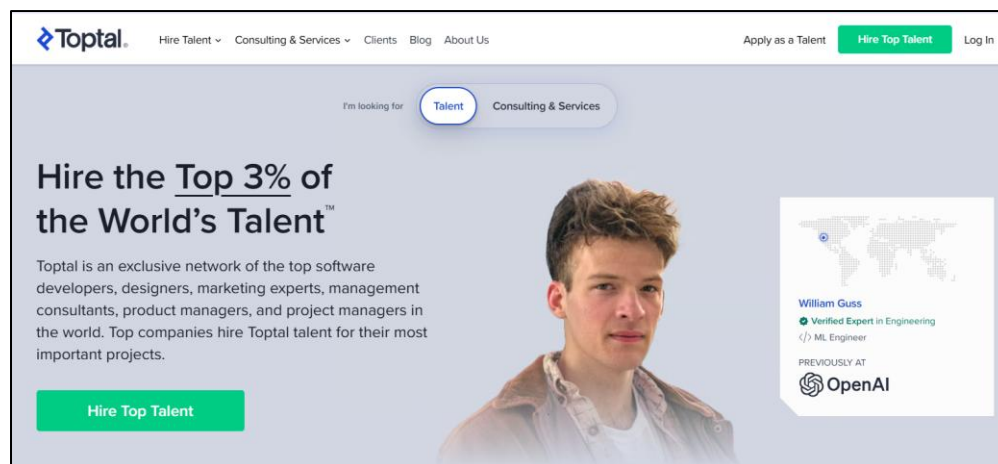


Figure 31: Toptal

8.3.5 Project 5 – Guru

Guru differentiates itself as a freelance platform by offering a balanced combination of flexibility, value, security, and continuous support for both employers and freelancers. The system is designed to accommodate various working styles, whether users operate independently, collaborate with agencies, or represent small and large organizations. Guru emphasizes affordability by

providing a wide range of project management and collaboration tools while maintaining comparatively low service fees, ensuring cost efficiency for all users. To promote trust and reliability, the platform includes SafePay payment protection, allowing freelancers to feel assured about receiving payments and employers to review work before funds are released, along with multiple secure payment options. Additionally, Guru offers 24/7 customer support, enabling users to receive timely assistance and resolve issues efficiently, regardless of their location. (guru, 2025)

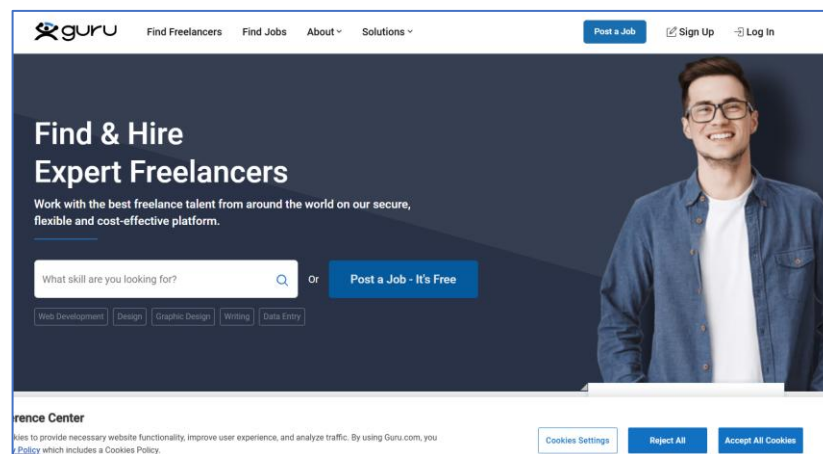


Figure 32: Guru

8.4 Considered Methodologies

8.4.1 Methodology 1 – Waterfall Model

The Waterfall model follows a linear, step-by-step approach where each development stage—such as requirements gathering, system design, coding, testing, and deployment—is completed before moving to the next. While it ensures well-documented processes and clarity, it is quite inflexible. For a platform like the freelancer–SME matchmaking system, requirements may change as features like secure login, project posting, and messaging are refined based on user feedback. Adapting to these changes would require revisiting previous stages, leading to delays and additional work. Because of this rigidity and difficulty handling evolving requirements, the Waterfall model is not ideal for this project.

8.4.2 Methodology 2 – V-Model

The V-Model emphasizes validation and verification at every stage of development. It improves system reliability and ensures thorough testing, but it remains rigid and sequential. In this project, where features may need adjustments—such as role-based access or communication tools—the V-Model does not easily accommodate changes without significant rework. Its inflexibility makes it unsuitable for a project that requires iterative development and frequent refinements.

8.4.3 Methodology 3 – Spiral Model

The Spiral model combines iterative development with risk assessment, focusing on minimizing potential project risks through repeated cycles. Although effective for large, complex, and high-risk projects, it is resource-heavy and complicated. For a student-led academic project like this, the Spiral model would be unnecessarily complex and could slow down progress. Its focus on risk management and extensive documentation makes it less practical for timely and efficient development of this platform.

8.4.4 Methodology 4 – Prototyping Model

The Prototyping model allows developers to build early versions of the system to gather user feedback. While this helps clarify requirements and improves user satisfaction, over-reliance on prototypes can lead to weak documentation and an unclear system structure. For this platform, excessive prototyping could cause inconsistency in implementing critical features such as secure project submissions, bidding, and freelancer–SME interactions. Maintaining a clear and well-documented system is essential, so the Prototyping model alone is not the best choice.

8.4.5 Methodology 5 – Agile Methodology

Agile methodology supports incremental development, continuous feedback, and adaptability to changing requirements. Features such as user registration, project management, bidding, communication tools, and secure authentication can be developed step by step in Agile sprints. This approach allows the team to respond

to feedback from SMEs, freelancers, and supervisors throughout the development process. Agile also promotes teamwork, frequent testing, and gradual delivery of functional modules, making it the most appropriate methodology for building the freelancer–SME matchmaking platform.

8.5 Requirement Gathering

Key functionalities such as account registration, secure login, role-based access control, project listing, bidding mechanisms, and communication features were prioritized according to project feasibility and time constraints. In addition, quality-related requirements—such as security, ease of use, system performance, and future scalability—were considered to support a reliable and user-friendly platform.

The results of this phase directly informed the development of the Software Requirements Specification (SRS), the product backlog, and the pre-survey documentation, all of which provided clear direction for system architecture design and sprint-based development activities.

8.6 SRS

8.6.1 Introduction

8.6.1.1 Intended audience

- Software developers involved in system implementation
- Project supervisors and academic evaluators
- System analysts, designers, and testers
- Academic reviewers and assessors

8.6.1.2 Definitions, acronyms and abbreviations

- **SME:** Small and Medium Enterprise
- **SRS:** Software Requirements Specification
- **UI:** User Interface
- **JWT:** JSON Web Token
- **CRUD:** Create, Read, Update, Delete

8.6.2 Overall Description

8.6.2.1 Product Perspective

The platform is designed as an independent web application built using modern development technologies. It follows a client–server architecture, consisting of a user-friendly frontend, a backend that manages application logic, and a database responsible for storing system data.

8.6.2.2 Product Functions

- Secure user registration and login
- Role-based access for SMEs and freelancers
- Project creation and management for SMEs
- Project search, viewing, and bidding for freelancers
- Secure communication between users
- User profile management
- Basic system administration and monitoring

8.6.2.3 User class and characteristics

User Type	Description
SME	Creates project listings, reviews freelancer proposals, and communicates with service providers
Freelancer	Searches projects, submits bids, and manages personal profiles

Figure 33: user class and characteristics

8.6.2.4 Operating environment

- Accessed through standard web browsers such as Chrome, Firefox, and Edge
- Backend developed using Node.js and Express
- Database managed with MongoDB
- Compatible with Windows, Linux, and macOS operating systems

8.6.2.5 Design and implementation constraints

- Limited development timeframe

- Academic project limitations
- Restricted budget and resources
- Deployment within a local or controlled hosting environment

8.6.2.6 Assumptions and dependencies

- Users have stable internet connectivity
- Users possess basic computer literacy
- The system depends on external libraries and frameworks such as JWT, bcrypt, and MongoDB

8.6.3 Functional Requirements

- Registration

Req. ID	Requirement Description		Priority	Complexity
FR 1.	User Registration		High	Medium
		The system shall allow new users, including SMEs and Freelancers, to create an account in order to use the platform.		
		A registration interface shall be provided where users can enter personal details and login credentials.		
		Users shall be required to submit mandatory		

		information such as full name, email address, selected role, and password.		
		The user shall submit their credentials for authentication.		
		The system shall verify that all required fields are properly completed before proceeding.		
		User inputs shall be validated to ensure correctness, including checking for duplicate email addresses.		
		User passwords shall be securely encrypted prior to being stored in the database.		
		Upon successful validation, the system shall create the user account and display a confirmation of successful registration.		

		If any validation errors occur, the system shall notify the user with appropriate error messages.		
--	--	---	--	--

Table 3: Functional Requirement of registration

- Login

Req. ID	Requirement Description		Priority	Complexity
FR 2.	User Login		High	Medium
		System Requirement		
		The system shall permit registered users to access the platform using valid login credentials.		
		A login interface shall be displayed requesting the user's email address and password.		
		The user shall submit their credentials for authentication.		
		The user shall submit their credentials for authentication.		
		The system shall verify the submitted		

		credentials against stored user records.		
		Upon successful authentication, the system shall create a secure session or authentication token.		
		Users shall be redirected to their respective dashboards based on assigned roles.		
		If authentication fails, the system shall inform the user by displaying an appropriate error message.		

Table 4: Login functional requirements

- Profile Management

Req. ID	Requirement Description	Priority	Complexity
FR 3.	Profile Management	High	Medium
FR 3.1	SME Profile Management		
	System Requirement		
	The system shall enable Small and Medium Enterprises (SMEs) to create and maintain		

FR 3.2		organizational profile information.		
		SME profiles shall contain essential business details including industry category, team size, previous project experience, preferred technology stack, and expected budget range.		
		The system shall clearly indicate the KYC verification status on SME profiles to enhance trust and transparency.		
		SMEs shall be permitted to modify and update their profile details whenever required.		
	Freelancer Profile Management			
		System Requirement		
		The system shall allow freelancers to develop comprehensive professional profiles showcasing their expertise.		
		Freelancer profiles shall include skill sets, portfolio materials, pricing details such as hourly or project-based rates, weekly		

		availability, and relevant social media links.		
		The system shall provide support for uploading portfolio resources, including images, PDF documents, and external links.		
		Freelancer profiles shall display ratings and feedback received from SMEs upon project completion.		

Table 5: Profile Management functional requirements

- Project Management

Req. ID	Requirement Description	Priority	Complexity
---------	-------------------------	----------	------------

FR 4.	Project Management	High	Medium										
FR 4.1	Project Posting												
	<table><tr><td></td><td>System Requirement</td></tr><tr><td></td><td>The system shall provide SMEs with the ability to create and publish new project listings on the platform.</td></tr><tr><td></td><td>Each project listing shall contain essential information, including required skill sets, expected experience level, project budget, submission deadline, and a detailed project overview.</td></tr><tr><td></td><td>Before a project is published, the system shall verify that all required fields are correctly completed and meet validation criteria.</td></tr><tr><td></td><td>Once approved, the posted projects shall be made visible to qualified freelancers for browsing and application.</td></tr></table>		System Requirement		The system shall provide SMEs with the ability to create and publish new project listings on the platform.		Each project listing shall contain essential information, including required skill sets, expected experience level, project budget, submission deadline, and a detailed project overview.		Before a project is published, the system shall verify that all required fields are correctly completed and meet validation criteria.		Once approved, the posted projects shall be made visible to qualified freelancers for browsing and application.		
		System Requirement											
		The system shall provide SMEs with the ability to create and publish new project listings on the platform.											
		Each project listing shall contain essential information, including required skill sets, expected experience level, project budget, submission deadline, and a detailed project overview.											
		Before a project is published, the system shall verify that all required fields are correctly completed and meet validation criteria.											
	Once approved, the posted projects shall be made visible to qualified freelancers for browsing and application.												
FR 4.2	Project Control												
	<table><tr><td></td><td>System Requirement</td></tr></table>		System Requirement	Medium	Low								
	System Requirement												

		The system shall enable SMEs to manage their posted projects by allowing them to close or cancel projects when necessary.		
		SMEs shall have the option to extend project deadlines through a simple and efficient action		
		Any changes made to the project status or timeline shall be communicated to relevant freelancers through system notifications.		

Table 6: Project Management SRS

- Browse and Apply

Req. ID	Requirement Description	Priority	Complexity
FR 6.	Browse and Apply	High	Medium
	System Requirement		
	The system shall enable freelancers to browse all currently available projects posted by SMEs on the platform.		
	Freelancers shall be provided with filtering tools		

		that allow projects to be refined based on criteria such as budget range, required skills, project deadline, and overall complexity.		
		Each project shall be presented in a clear and structured format, displaying relevant details to help freelancers evaluate suitability.		
		The system shall allow freelancers to submit proposals for selected projects, including a bid amount, a written proposal description, and optional supporting files or portfolio items.		
		All proposal submissions shall undergo validation to ensure completeness and accuracy.		
		The system shall monitor and update the status of submitted proposals throughout the selection process.		

--	--	--	--

Table 7: Browse and Apply SRS

- Bidding and Matchmaking

Req. ID	Requirement Description	Priority	Complexity
FR 7.	Bidding and Matchmaking	High	Medium
	System Requirement		
	The system shall provide SMEs with access to all proposals submitted for their projects.		
	SMEs shall be able to organize and compare proposals using sorting options such as proposed cost, freelancer ratings, professional experience, and response time.		
	The system shall support the shortlisting of freelancers to assist SMEs in the final selection process.		
	During evaluation, SMEs shall be able to view freelancer profiles, portfolios, and previous reviews.		
	Freelancers shall be able to track the progress of their proposals through clearly		

		defined status updates, including viewed, shortlisted, accepted, or rejected.		
		The system shall notify freelancers whenever a proposal status is updated.		

Table 8: Bidding and matchmaking SRS

- Escrow-Based Payment System

Req. ID	Requirement Description		Priority	Complexity
FR 8.	Escrow-Based Payment System		High	High
		System Requirement		
		The system shall require SMEs to deposit the full project payment into an escrow account before the freelancer is permitted to begin work.		
		Work initiation shall be restricted until escrow funding is successfully confirmed.		
		Upon completion and submission of final deliverables, the system shall release the payment to the freelancer only after SME approval.		

		If submitted work is rejected, the system shall initiate a dispute resolution process.		
		The system shall integrate with the eSewa payment gateway to handle all financial transactions securely.		
		Additionally, the system shall record key payment including deposit, submission, approval, and fund release.		

Table 9: Escrow-Based Payment System SRS

- Communication Module

Req. ID	Requirement Description	Priority	Complexity
FR 9.	Communication Module	Medium	Medium
	System Requirement		
	The system shall provide a dedicated communication space for each project, allowing direct interaction between SMEs and freelancers.		
	All project-related messages shall be securely stored and displayed in chronological order for easy reference.		

		The communication feature shall support either real-time messaging or threaded conversations based on system design.		

Table 10: Communication Module SRS

- Review and Rating System

Req. ID	Requirement Description	Priority	Complexity
---------	-------------------------	----------	------------

FR 10.	Review and Rating System	Medium	Low												
	<table><tr><td></td><td>System Requirement</td></tr><tr><td></td><td>The system shall allow SMEs to assess freelancers using a structured rating framework based on predefined performance factors.</td></tr><tr><td></td><td>Freelancers shall be able to submit ratings and feedback for SMEs after project completion.</td></tr><tr><td></td><td>Submitted reviews and ratings shall be publicly visible on user profiles.</td></tr><tr><td></td><td>The system shall restrict review submission to completed projects only.</td></tr><tr><td></td><td>Ratings shall contribute to overall user credibility and platform transparency.</td></tr></table>		System Requirement		The system shall allow SMEs to assess freelancers using a structured rating framework based on predefined performance factors.		Freelancers shall be able to submit ratings and feedback for SMEs after project completion.		Submitted reviews and ratings shall be publicly visible on user profiles.		The system shall restrict review submission to completed projects only.		Ratings shall contribute to overall user credibility and platform transparency.		
	System Requirement														
	The system shall allow SMEs to assess freelancers using a structured rating framework based on predefined performance factors.														
	Freelancers shall be able to submit ratings and feedback for SMEs after project completion.														
	Submitted reviews and ratings shall be publicly visible on user profiles.														
	The system shall restrict review submission to completed projects only.														
	Ratings shall contribute to overall user credibility and platform transparency.														

Table 11: Rating and review SRS

- Admin Dashboard

Req. ID	Requirement Description	Priority	Complexity
FR 11.	Admin Dashboard	High	Medium
	System Requirement		
	The system shall provide administrators with a centralized dashboard for managing users and platform activities.		
	Administrators shall have the authority to activate or deactivate user accounts when necessary.		
	The system shall allow administrators to review and approve KYC documentation submitted by users.		
	Administrators shall be able to monitor projects across all statuses, including active, completed, cancelled, and disputed.		
	The system shall enable administrators to evaluate disputes and make final decisions regarding escrow fund distribution.		

--	--	--	--

Table 12: Admin Dashboard SRS

- AI-Based Project Estimation

Req. ID	Requirement Description	Priority	Complexity
FR 12.	AI- Based Project Estimation	Medium	High
	System Requirement		
	The system shall use artificial intelligence to estimate project costs and expected completion timelines.		
	Estimates shall be generated using factors such as project type, complexity, historical records, and market trends.		
	The system shall present recommended budget ranges and timelines to SMEs during project creation.		
	The estimation feature shall assist SMEs in making informed financial and scheduling decisions.		
	The system shall alert SMEs when proposed budgets or		

		deadlines are considered unrealistic.		

Table 13: AI-based estimation SRS

8.6.4 Non-Functional Requirements

Non-functional requirements define the quality standards of the system rather than its operational features. These requirements describe how effectively the system performs under various conditions and play a crucial role in determining user satisfaction and system dependability. A system that meets functional goals but ignores non-functional aspects may still fail to achieve acceptance. The key quality attributes identified for the Freelancer–SME Matchmaking Platform are presented below.

Reliability

Attribute ID	Requirement Description	Priority	Complexity
R.1	The application shall function reliably without unexpected crashes during normal operations such as user authentication and project management.	High	High
R.2	The system shall maintain stable performance and	Maximum	Normal

	provide timely responses without noticeable delays during user interaction.		
--	---	--	--

Table 14: Non-functional requirement – reliability

- Security

Attribute ID	Requirement Description	Priority	Complexity
R.1	The system shall protect all data transmissions using secure and encrypted communication mechanisms.	Maximum	High
R.2	Access to system features shall be restricted to users who have been properly verified and authorized according to their roles.	Maximum	Normal

Table 15: Non-functional requirement – Security

- Maintainability

Attribute ID	Requirement Description	Priority	Complexity

R.1	The application shall deliver smooth and efficient performance, ensuring an improved user experience through responsive interface design.	Maximum	High
-----	---	---------	------

Table 16: Non-Functional requirement – performance

8.6.5 External Interface Requirements

8.6.6 User Interface

The system shall provide a browser-based graphical interface that allows users to interact with the platform efficiently. The interface will be designed to adapt seamlessly to different screen resolutions and device sizes, ensuring consistent usability across desktops, laptops, tablets, and mobile devices.

8.6.7 Hardware Interface

The application does not depend on any specialized hardware components. It can operate on standard computing devices that support a modern web browser and an internet connection.

8.6.8 Software Interface

The system shall use MongoDB as the primary database solution for storing and managing application data. Node.js will be utilized as the runtime environment to support backend services and server-side processing.

8.7 Sprint 1 / Prototype 1 – Login and Register

8.7.1 Sprint Planning

For Sprint 1, the objectives and boundaries were carefully outlined, with emphasis placed on developing the fundamental authentication features of the system. This

sprint primarily covered user registration, login functionality, and basic role identification. Tasks were organized according to priority and technical dependencies, and achievable timelines were set to maintain steady progress. The sprint backlog was prepared to ensure that all activities remained aligned with the system requirements as well as the academic goals of the project.

8.7.2 Design

The design stage involved preparing both conceptual and detailed design elements to clearly define the authentication process. High-level Use Case diagrams were created to present an overall view of user interactions related to login and registration. These were supported by expanded Use Case descriptions that explained each step of the interaction, including system responses and alternative scenarios such as incorrect credentials or incomplete input.

Activity Diagrams were used to represent the sequence of actions and decision points involved in authentication, while Sequence Diagrams illustrated the communication flow between the user interface, application logic, and database components. A Level 1 Data Flow Diagram was also developed to show how data is processed and transferred during login and registration operations. In addition, supporting visuals such as flowcharts and mind maps were prepared to simplify complex logic and provide a clearer understanding of the system workflow, ensuring a solid design foundation before implementation.

8.7.2.1 High Level Use case

1) Login

Actor: SME / Freelancer

Description:

Allows users to securely access the system using their credentials.

2) Register

Actor: SME / Freelancer

Description:

Allows new users to create an account on the platform.

3) Update Profile

Actor: SME / Freelancer Description:

Allows users to modify personal or business information.

4) Post Project

Actor: SME

Description:

Allows SMEs to publish project requirements for freelancers.

5) Manage Posted Projects

Actor: SME

Description:

Allows SMEs to manage their posted projects.

6) Review Bids

Actor: SME

Description:

Allows SMEs to evaluate bids submitted by freelancers.

7) Select Freelancer

Actor: SME

Description:

Allows SMEs to hire a freelancer for a project.

8) Browse Projects

Actor: Freelancer

Description:

Allows freelancers to view available projects.

9) Submit Bid

Actor: Freelancer

Description:

Allows freelancers to apply for projects.

10) Work on Assigned Tasks

Actor: Freelancer

Description:

Allows freelancers to work on assigned project tasks.

11) Send Deliverables

Actor: Freelancer

Description:

Allows freelancers to submit completed work.

12) Request Payment Release

Actor: Freelancer Description:

Allows freelancers to request payment after work submission.

13) Release Payment

Actor: SME

Description:

Allows SMEs to release payment after verifying work.

14) Raise Dispute

Actor: SME / Freelancer

Description:

Allows users to raise disputes if issues occur.

15) Chat with SME

Actor: SME / Freelancer

Description:

Allows users to communicate during a project.

8.7.2.2 Expanded Use case

1) User Login

Actors: SME, Freelancer

Description:

This use case enables registered users to gain secure access to the system by providing valid login credentials.

Preconditions:

- The user must already have a registered account.
- The account must be active and not restricted.

Main Flow:

1. The user navigates to the login page.
2. The system displays fields for email and password.
3. The user enters the required credentials.
4. The system verifies the information against stored records.
5. Upon successful verification, the system authenticates the user.
6. The user is redirected to the appropriate dashboard.

Alternative Flow:

If incorrect credentials are entered, an error message is displayed.

Postconditions:

The user is successfully logged into the system.

2) User Registration

Actors: SME, Freelancer

Description:

This use case allows new users to create an account on the platform.

Preconditions:

The user must not already be registered with the same email address.

Main Flow:

1. The user accesses the registration page.
2. The system presents a registration form.
3. The user enters personal details and selects a role.
4. The system validates the provided information.
5. The password is securely encrypted and stored.
6. The system confirms successful account creation.

Alternative Flow:

If validation fails, the system prompts the user to correct the errors.

Postconditions:

A new user account is created and stored in the system.

3) Update Profile

Actors: SME, Freelancer

Description:

This use case allows users to modify their personal or professional profile information.

Preconditions:

The user must be logged in.

Main Flow:

1. The user accesses the profile management section.
2. Existing profile details are displayed.
3. The user updates relevant information.
4. The system validates the changes.
5. The updated profile information is saved.

Postconditions:

Profile details are successfully updated.

4) Post Project

Actor: SME

Description:

This use case enables SMEs to publish project requirements for freelancers.

Preconditions:

The SME must be logged in.

Main Flow:

1. The SME selects the option to post a new project.
2. The system displays a project creation form.
3. The SME enters project details such as skills, budget, and deadline.
4. The system validates the entered information.
5. The project is published and made visible to freelancers.

Postconditions:

A new project listing becomes available on the platform.

5) Manage Posted Projects

Actor: SME

Description:

This use case allows SMEs to manage their previously posted projects.

Preconditions:

The SME must have at least one active project.

Main Flow:

1. The SME views a list of posted projects.
2. A specific project is selected.
3. The SME updates project status, closes, cancels, or extends deadlines.
4. The system applies and saves the changes.

Postconditions:

Project status is updated accordingly.

6) Review Bids

Actor: SME

Description:

This use case enables SMEs to evaluate proposals submitted by freelancers.

Preconditions:

Freelancers must have submitted bids for the project.

Main Flow:

1. The SME accesses the list of received bids.
2. The system displays bid details and freelancer profiles.
3. The SME compares bids based on provided criteria.
4. Preferred freelancers may be shortlisted.

Postconditions:

Bids are reviewed and shortlisted where applicable.

7) Select Freelancer

Actor: SME

Description:

This use case allows SMEs to choose a freelancer for project execution.

Preconditions:

Bids must have been reviewed.

Main Flow:

1. The SME selects a freelancer from the shortlisted bids.
2. The system confirms the selection.
3. The project status is updated to assigned.
4. The selected freelancer is notified.

Postconditions:

The freelancer is officially assigned to the project.

8) Browse Projects

Actor: Freelancer

Description:

This use case allows freelancers to explore available project listings.

Preconditions:

The freelancer must be logged in.

Main Flow:

1. The freelancer navigates to the project browsing section.
2. The system displays all open projects.
3. Filters may be applied to refine results.
4. Project details are viewed.

Postconditions:

Relevant project information is displayed.

9) Submit Bid

Actor: Freelancer

Description:

This use case enables freelancers to apply for projects.

Preconditions:

The freelancer must be logged in.

Main Flow:

1. The freelancer selects a project.
2. A bid submission form is displayed.
3. The freelancer enters bid details and submits the proposal.
4. The system validates and records the bid.

Postconditions:

The proposal is successfully submitted.

10) Work on Assigned Tasks

Actor: Freelancer

Description:

This use case allows freelancers to work on assigned project tasks.

Preconditions:

The freelancer must be assigned to a project.

Main Flow:

1. The freelancer accesses the project workspace.
2. Project requirements are reviewed.
3. Assigned tasks are carried out.

Postconditions:

Project work progresses.

11) Send Deliverables

Actor: Freelancer

Description:

This use case allows freelancers to submit completed project work.

Preconditions:

Assigned tasks must be completed.

Main Flow:

1. The freelancer uploads project deliverables.
2. The system validates file formats and size.
3. The SME is notified of submission.

Postconditions:

Deliverables are successfully submitted.

12) Request Payment Release

Actor: Freelancer

Description:

This use case allows freelancers to request payment after submitting deliverables.

Preconditions:

Work must be submitted for review.

Main Flow:

1. The freelancer submits a payment release request.
2. The system notifies the SME.

Postconditions:

Payment request is recorded.

13) Release Payment

Actor: SME

Description:

This use case allows SMEs to release payment after verifying completed work.

Preconditions:

Deliverables must be reviewed and approved.

Main Flow:

1. The SME reviews the submitted work.
2. Approval is granted.
3. The system releases escrow funds.

Postconditions:

Payment is transferred to the freelancer.

14) Raise Dispute

Actors: SME, Freelancer

Description:

This use case allows users to report issues related to a project.

Preconditions:

An active or completed project must exist.

Main Flow:

1. The user submits a dispute request.
2. The system records dispute details.
3. Administrators are notified.

Postconditions:

A dispute case is created.

15) Chat with SME / Freelancer

Actors: SME, Freelancer

Description:

This use case enables communication between project participants.

Preconditions:

The project must be active.

Main Flow:

1. The user opens the project communication channel.
2. Messages are exchanged.
3. The system stores all messages securely.

Postconditions:

Communication history is saved.

8.8 Pre-survey response

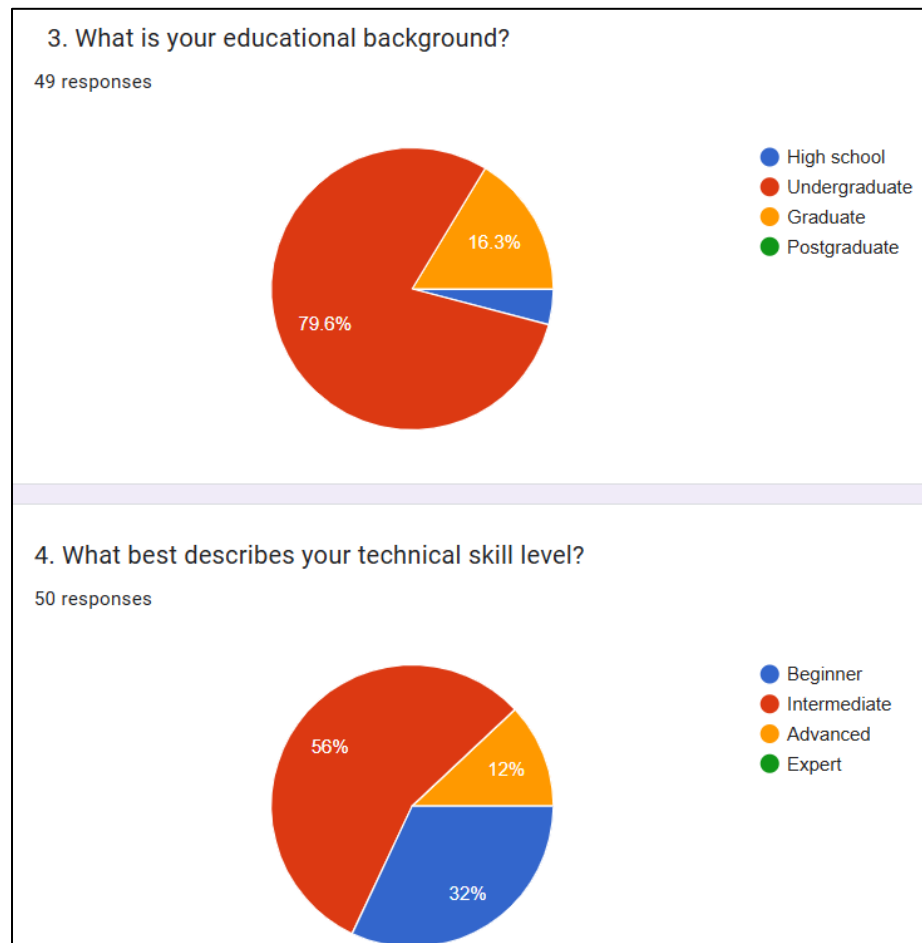


Figure 34: Pre survey question 3 and 4 response

Result: Among 49 respondents, 79.6% held undergraduate degrees, 16.3% had graduate degrees, a few had completed high school, and no postgraduate degree holders participated.

Out of 50 participants, 56% described themselves as intermediate users, 32% as beginners, and 12% as advanced. No one considered themselves an expert.

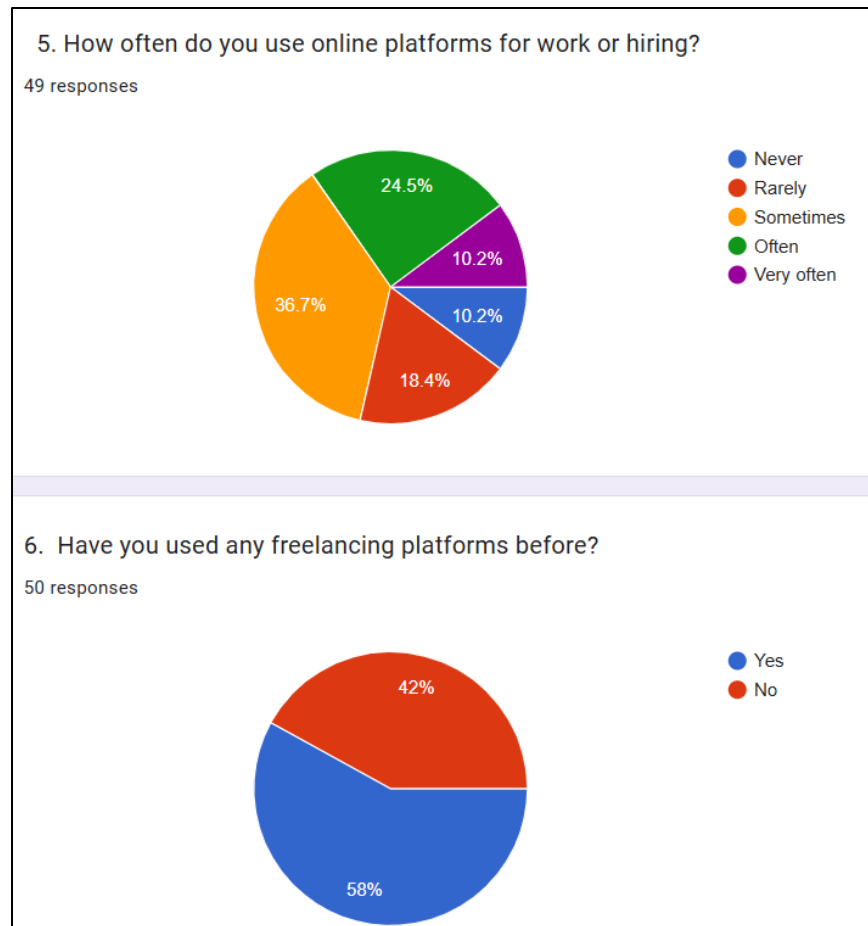


Figure 35: Pre survey question 5 and 6 response

Result:

From 49 responses, 36.7% reported using online platforms occasionally, 24.5% often, 10.2% very often, 18.4% rarely, and 10.2% never used them for work or hiring purposes.

Of 50 respondents, 58% had prior experience using freelancing platforms, while 42% had none.

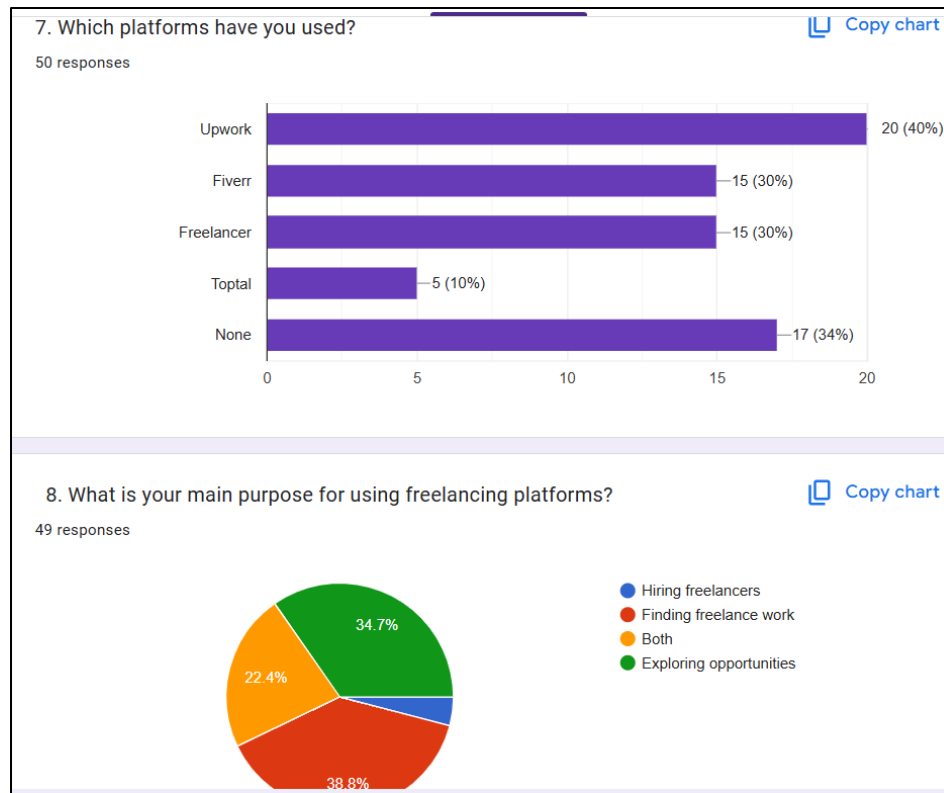


Figure 36: Pre survey question 7 and 8 response

Result:

Upwork was the most popular (40%), followed by Fiverr and Freelancer (30% each), and Toptal (10%). About 34% of participants had not used any platform.

Among 49 participants, 38.8% used the platforms to find freelance work, 34.7% for exploring opportunities, 22.4% for both hiring and finding work, and 4.1% solely for hiring freelancers.

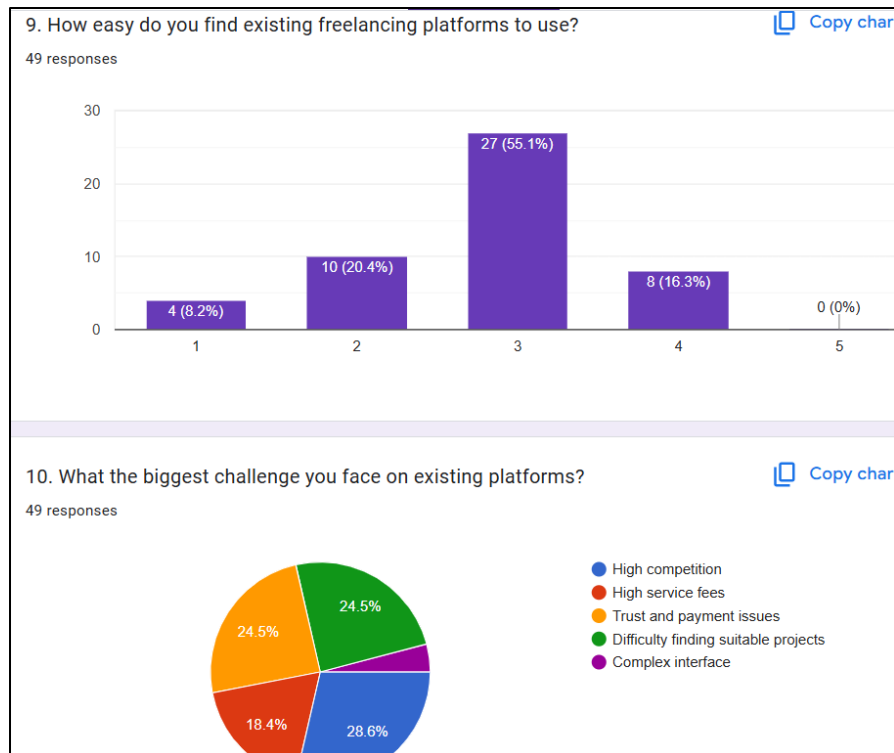


Figure 37: Pre survey question 9 and 10 response

Result:

Out of 49 responses, 55.1% rated ease of use as neutral (3), 20.4% found it somewhat difficult (2), 16.3% somewhat easy (4), and 8.2% very difficult (1). No one rated it as very easy (5).

The main challenges reported by 49 respondents were high competition (28.6%), trust and payment concerns (24.5%), difficulty finding suitable projects (24.5%), high service fees (18.4%), and complex interfaces (4.1%).

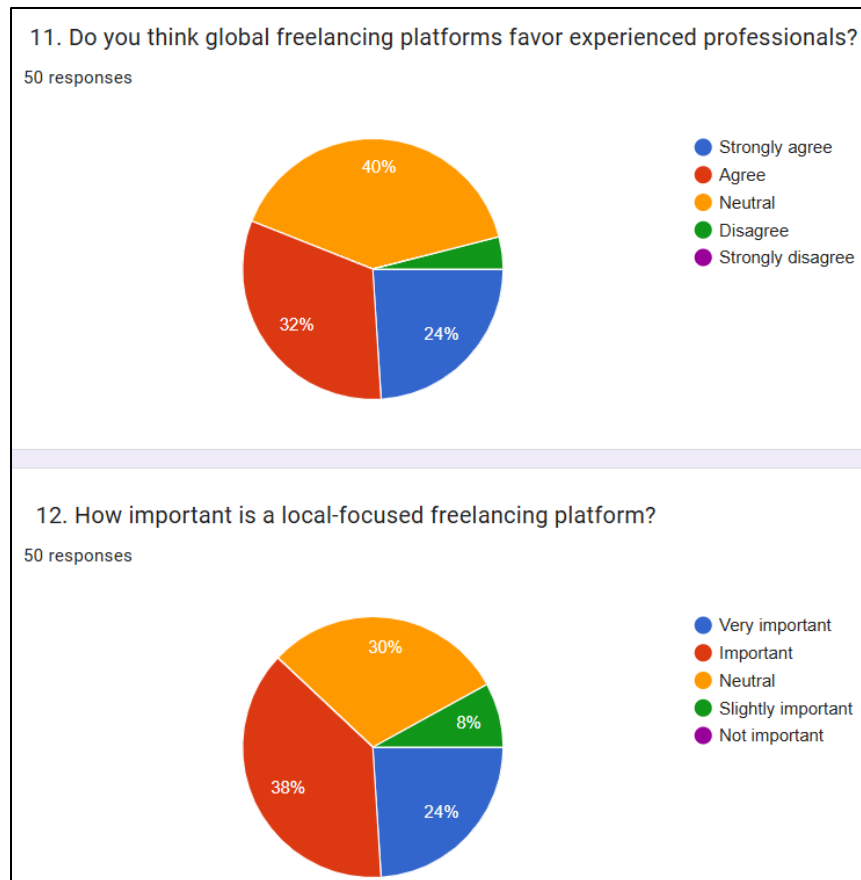


Figure 38: Pre survey question 11 and 12 response

Result:

56% agreed that global freelancing platforms favour experienced professionals (24% strongly agree, 32% agree), 40% were neutral, and only 4% disagreed, showing a perception that experience is an advantage.

62% of respondents considered a local-focused freelancing platform important (24% very important, 38% important), 30% were neutral, and 8% slightly important, highlighting interest in local platforms.

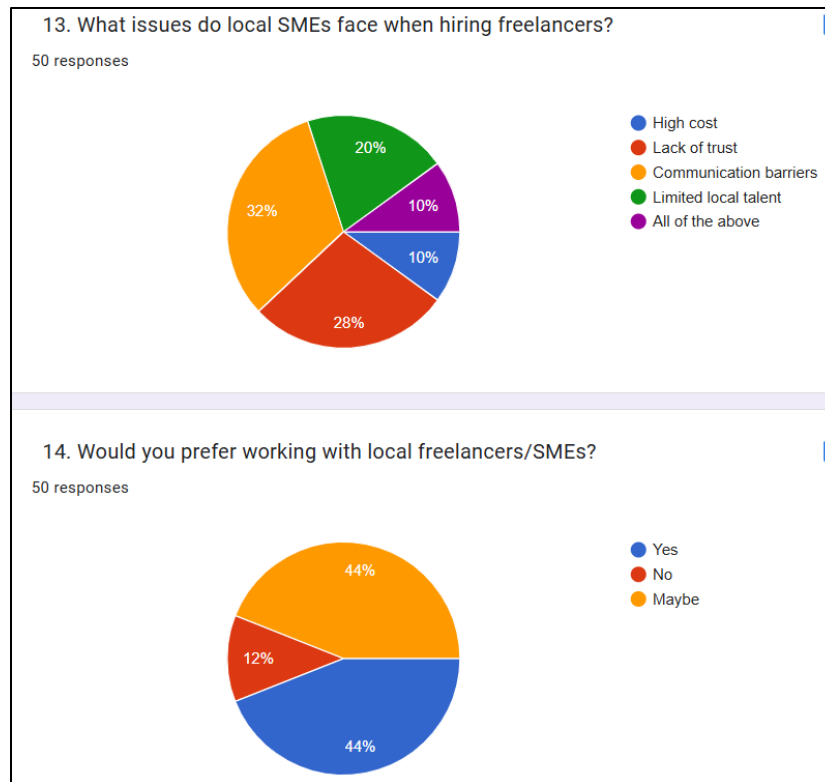


Figure 39: Pre survey question 13 and 14 response

Result:

Communication barriers were most cited (32%), followed by lack of trust (28%), limited local talent (20%), and high costs or all of the above (10% each).

Responses were evenly split: 44% said yes, 44% maybe, and 12% no, reflecting openness but some uncertainty regarding local collaboration.

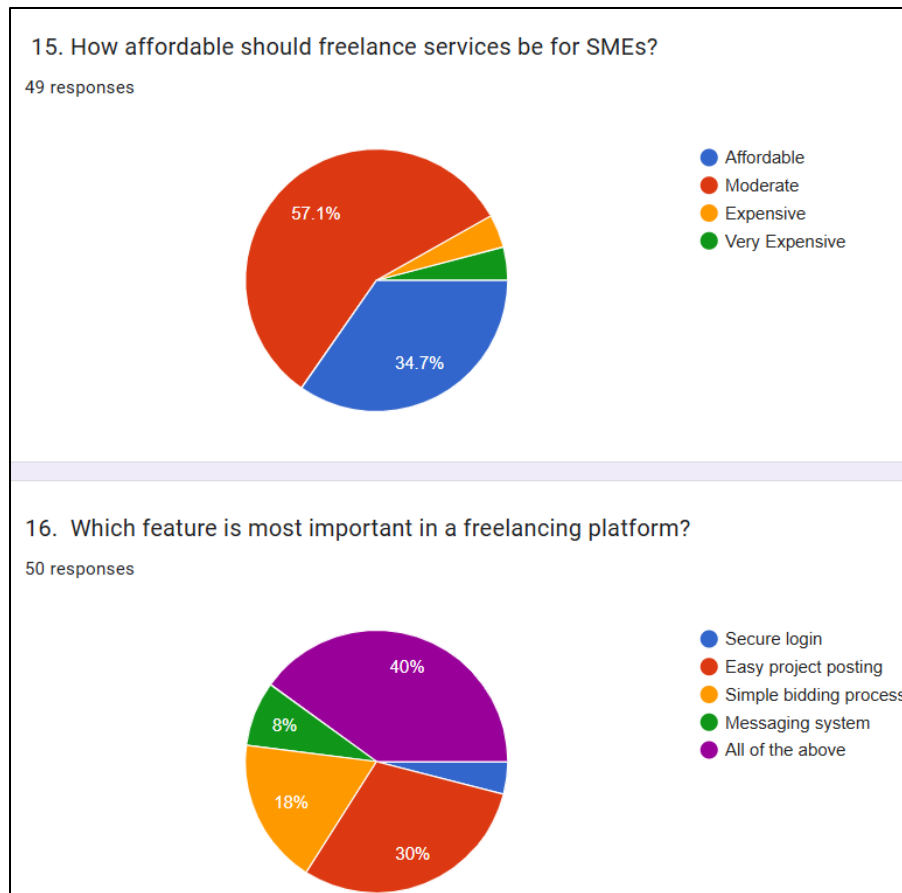


Figure 40: Pre survey question 15 and 16 response

Result:

57.1% preferred moderate pricing, 34.7% favored affordable rates, and a small percentage opted for expensive or very expensive services.

40% valued all features combined, 30% prioritized easy project posting, 18% preferred a simple bidding process, and 8% emphasized messaging systems.

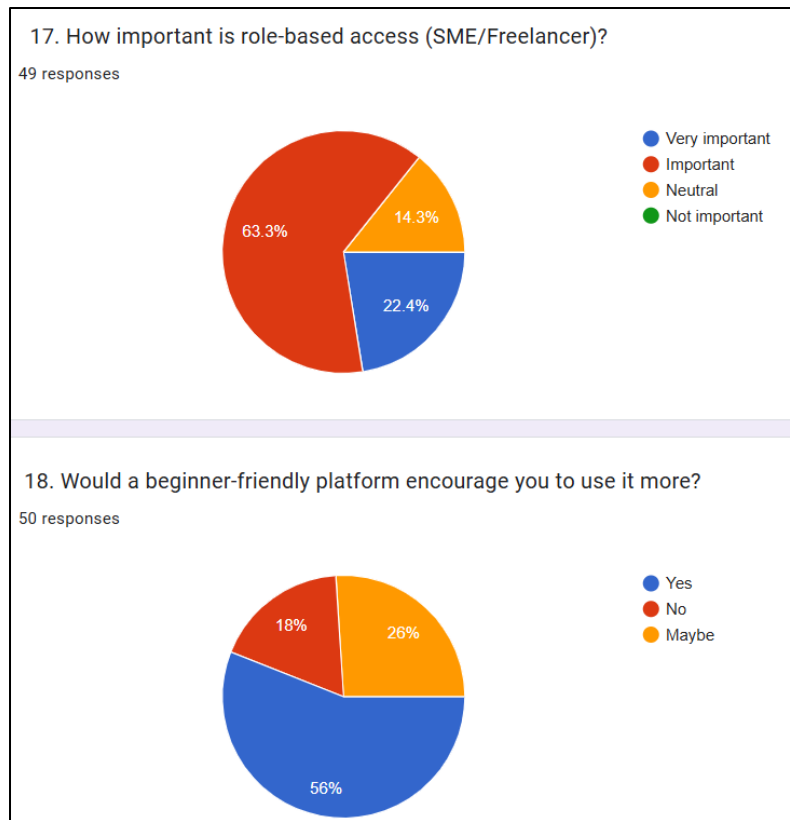


Figure 41: Pre survey question 17 and 18 response

Result:

85.7% considered role differentiation important (22.4% very important, 63.3% important), with 14.3% neutral, showing strong demand for role-based access.

56% said a beginner-friendly platform would encourage usage, 26% maybe, and 18% no, indicating that beginner-friendliness is a key motivator.

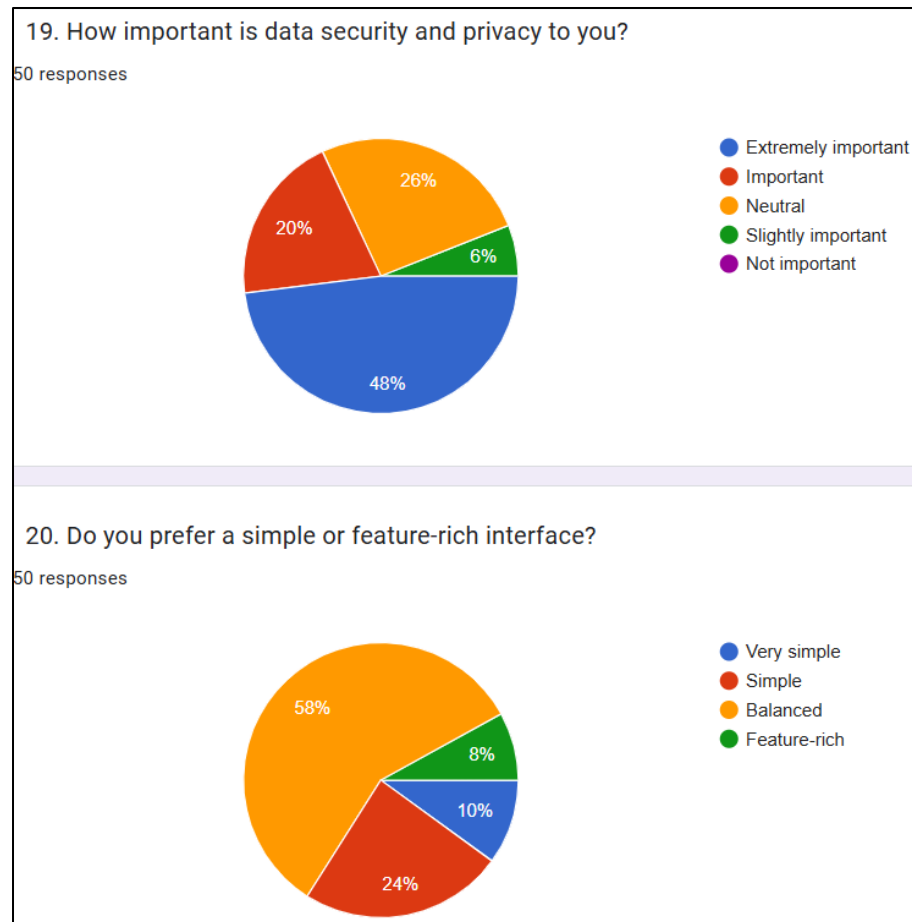


Figure 42: Pre survey question 19 and 20 response

Result:

68% rated data security and privacy as highly important (48% extremely important, 20% important), 26% were neutral, and 6% slightly important.

58% preferred a balanced design, 24% a simple interface, 10% very simple, and 8% feature-rich, indicating a preference for moderate functionality.

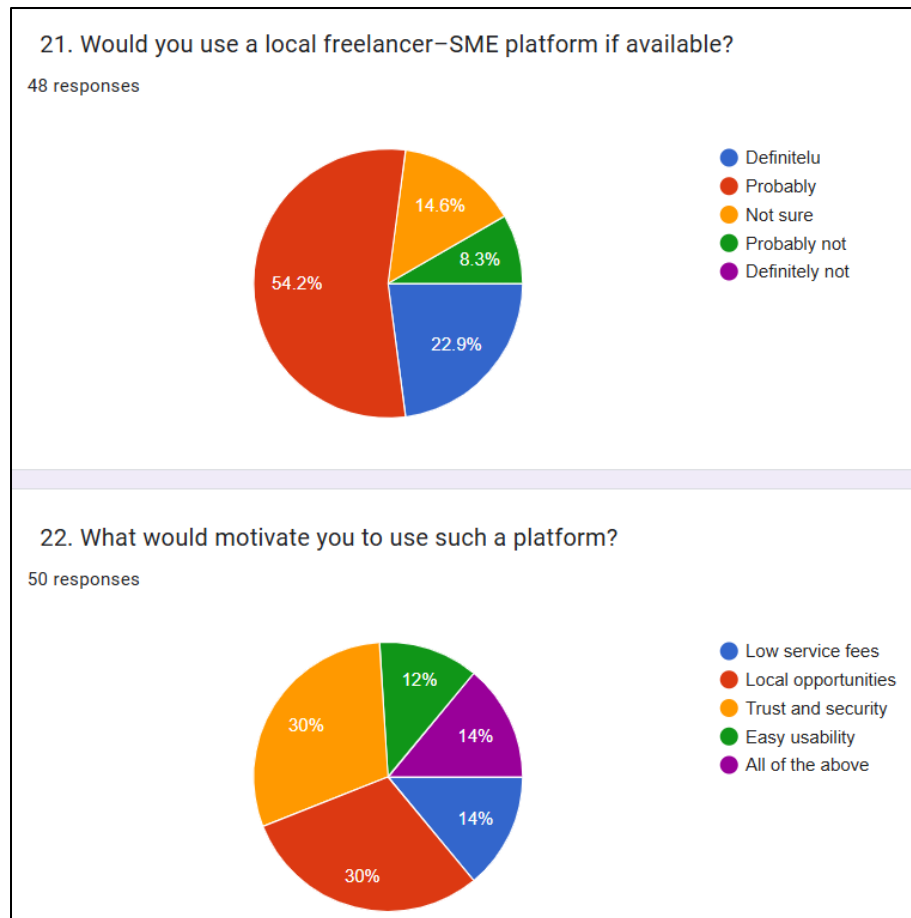


Figure 43: Pre survey question 21 and 22 response

Result:

77.1% expressed interest in using a local freelancer-SME platform (22.9% definitely, 54.2% probably), 14.6% were unsure, and 8.3% probably not.

Trust and security (30%) and local opportunities (30%) were the main motivators, followed by “all of the above” (14%), low service fees (14%), and easy usability (12%).

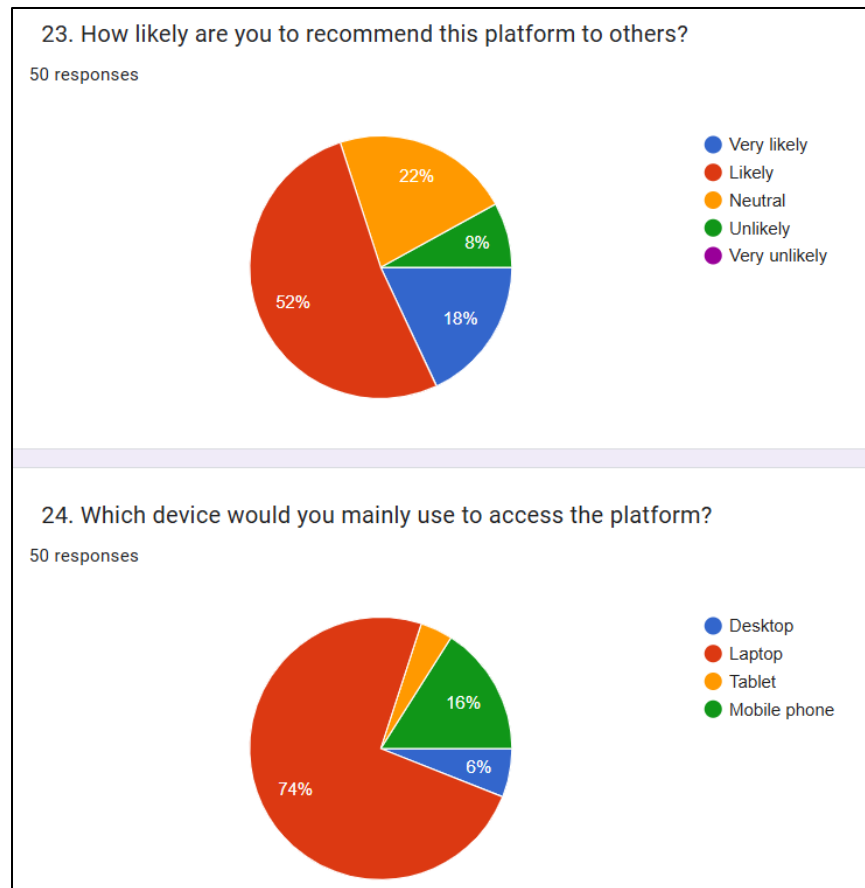


Figure 44: Pre survey question 23 and 24 response

Result:

70% would recommend the platform (52% likely, 18% very likely), 22% were neutral, and 8% were unlikely to recommend it.

Laptops were most preferred (74%), followed by mobile phones (16%), desktops (6%), and tablets (4%), suggesting a preference for portable computing devices.

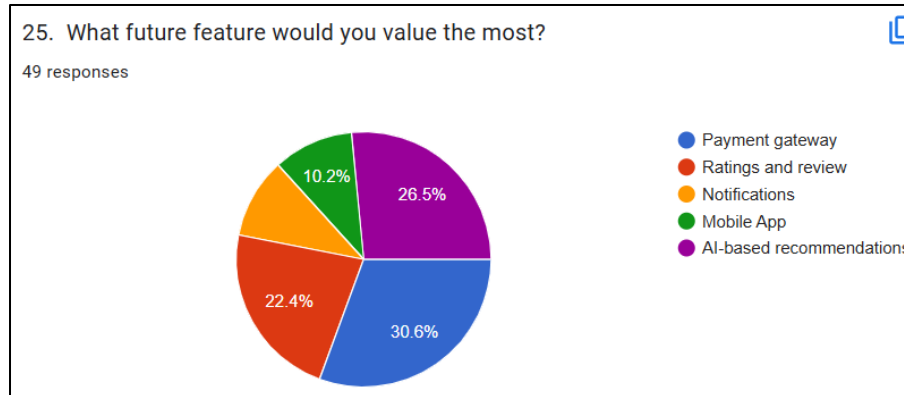


Figure 45: Pre survey question 25 response

Result:

Payment gateway was the top priority (30.6%), followed by AI-based recommendations (26.5%), ratings and review systems (22.4%), and mobile app development (10.2%), showing diverse feature expectations.

8.9 Development

8.9.1 Frontend

- Login.jsx

```
frontend > src > pages > Login.jsx > Login > handleLogin
1  import { useState } from "react";
2  import { Link, useNavigate } from "react-router-dom";
3  import "./Login.css";
4
5  export default function Login() {
6    const [form, setForm] = useState({ email: "", password: "" });
7    const navigate = useNavigate();
8
9    const handleChange = (e) => {
10     setForm({ ...form, [e.target.name]: e.target.value });
11   };
12
13   const handleLogin = async (e) => {
14     e.preventDefault();
15
16     if (!form.email || !form.password) {
17       alert("Please enter both email and password");
18       return;
19     }
20
21     try {
22       const res = await fetch("http://localhost:5000/api/auth/login", {
23         method: "POST",
24         headers: { "Content-Type": "application/json" },
25         body: JSON.stringify(form),
26       });
27
28       const data = await res.json();
29       if (!res.ok) return alert(data.message);
30
31       // Store JWT tokens
32       localStorage.setItem("accessToken", data.accessToken);
33       localStorage.setItem("refreshToken", data.refreshToken);
```

Figure 46: Login.jsx

```
4   <div className="auth-card">
5     <h2 className="auth-title">Sign in to your account</h2>
6     <p className="auth-subtitle">Welcome back. Please enter your details.</p>
7
8     <form onSubmit={handleLogin} className="auth-form">
9       <input
10        name="email"
11        type="email"
12        placeholder="Email address"
13        value={form.email}
14        onChange={handleChange}
15        required
16      />
17
18      <input
19        name="password"
20        type="password"
21        placeholder="Password"
22        value={form.password}
23        onChange={handleChange}
24        required
25      />
26
27      <button type="submit">Sign In</button>
28    </form>
29
30    <p className="signup-link">
31      Don't have an account? <Link to="/register">Sign Up</Link>
32    </p>
33  </div>
34</div>
35);
```

Figure 47: Login.jsx

- Login.css

```
ontend > src > pages > # Login.css > ...
1  /* Page background */
2  .auth-page {
3      min-height: 100vh;
4      display: flex;
5      justify-content: center;
6      align-items: center;
7      background: linear-gradient(135deg, #f7f1e8 0%, #f0e5d4 50%, #e8dbc1 100%);
8      font-family: "Segoe UI", Tahoma, sans-serif;
9  }
10
11 /* Card container */
12 .auth-card {
13     background-color: #ffffff;
14     width: 100%;
15     max-width: 460px;
16     padding: 40px 35px;
17     border-radius: 14px;
18     box-shadow: 0 25px 50px rgba(0, 0, 0, 0.12);
19     display: flex;
20     flex-direction: column;
21     align-items: center;
22 }
23
24 /* Titles */
25 .auth-title {
26     font-size: 24px;
27     color: #4a3728;
28     font-weight: 600;
29     margin-bottom: 6px;
30     text-align: center;
31 }
```

Figure 48: Login.css

```
frontend > src > pages > # Login.css > ...
73 .auth-form button {
74   background-color: #b08968;
75   color: #ffffff;
76   font-size: 15px;
77   font-weight: 600;
78   cursor: pointer;
79   transition: background 0.25s ease, transform 0.1s ease;
80 }
81
82 .auth-form button:hover {
83   background-color: #6b4f3f;
84 }
85
86 .auth-form button:active {
87   transform: scale(0.98);
88 }
89
90 /* Sign up link below form */
91 .signup-link {
92   margin-top: 18px;
93   font-size: 14px;
94   color: #8a7a64;
95   text-align: center;
96 }
97
98 .signup-link a {
99   color: #b08968;
100   text-decoration: none;
101   font-weight: 600;
102 }
103
104 .signup-link a:hover {
105   text-decoration: underline;
106 }
```

Figure 49: Login.css

- Register.jsx

```

frontend > src > pages > Register.jsx > Register
1  import { useState } from "react";
2  import { Link, useNavigate } from "react-router-dom";
3  import "./Register.css";
4
5  export default function Register() {
6    const [form, setForm] = useState({
7      firstName: "",
8      lastName: "",
9      fullName: "",
10     email: "",
11     password: "",
12     confirmPassword: "",
13     role: "Freelancer",
14   });
15
16   const navigate = useNavigate();
17
18   const handleChange = (e) => {
19     setForm({ ...form, [e.target.name]: e.target.value });
20   };
21
22   const handleRegister = async (e) => {
23     e.preventDefault();
24
25     if (!form.firstName || !form.lastName || !form.fullName || !form.email || !form.password || !form.confirmPassword) {
26       alert("All fields are required");
27       return;
28     }
29
30     if (form.password !== form.confirmPassword) {
31       alert("Passwords do not match");

```

Figure 50: Register.jsx

```

end > src > pages > Register.jsx > Register
export default function Register() {
  const handleRegister = async (e) => {
    try {
      const res = await fetch("http://localhost:5000/api/auth/register", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(form),
      });

      const data = await res.json();
      if (!res.ok) return alert(data.message);

      // Store JWT tokens
      localStorage.setItem("accessToken", data.accessToken);
      localStorage.setItem("refreshToken", data.refreshToken);

      alert("Registration successful");
      navigate("/dashboard"); // redirect to protected dashboard
    } catch {
      alert("Server error");
    }
  };

  return (
    <div className="register-page">
      <div className="register-card">
        <h2>Create your account</h2>
        <p className="register-subtitle">Join our SMe-freelancer matchmaking platform today</p>

        <form className="register-form" onSubmit={handleRegister}>
          <input
            name="firstName"
            placeholder="First Name"

```

Figure 51: Register.jsx

```
export default function Register() {  
  name="email"  
  type="email"  
  placeholder="Email address"  
  onChange={handleChange}  
  required  
/>  
<input  
  name="password"  
  type="password"  
  placeholder="Password"  
  onChange={handleChange}  
  required  
/>  
<input  
  name="confirmPassword"  
  type="password"  
  placeholder="Confirm Password"  
  onChange={handleChange}  
  required  
/>  
<select name="role" onChange={handleChange}>  
  <option value="Freelancer">Freelancer</option>  
  <option value="SME">SME</option>  
</select>  
<button type="submit">Create Account</button>  
</form>  
  
<p className="login-link">  
  Already have an account? <Link to="/login">Login</Link>  
</p>  
</div>  
</div>
```

Figure 52: Register.jsx

- Register.css

```
d > src > pages > # Register.css > .register-card
/* Page background */
.register-page {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  background: linear-gradient(135deg, #f7f1e8 0%, #f0e5d4 50%, #e8dbc1 100%);
  font-family: "Segoe UI", Tahoma, sans-serif;
}

/* Card container */
.register-card {
  background-color: #ffffff;
  width: 100%;
  max-width: 460px;
  padding: 40px 35px;
  border-radius: 14px;
  box-shadow: 0 25px 50px rgba(0, 0, 0, 0.12);
  display: flex;
  flex-direction: column;
  align-items: center;
}

/* Titles */
.register-card h2 {
  font-size: 24px;
  color: #4a3728;
  font-weight: 600;
  margin-bottom: 6px;
  text-align: center;
}

.register-subtitle {
```

Figure 53: Register.css

```
.register-subtitle {
  font-size: 14px;
  color: #8a7a64;
  margin-bottom: 28px;
  text-align: center;
}

/* Form */
.register-form {
  display: flex;
  flex-direction: column;
  width: 100%;
  gap: 14px;
}

/* Inputs and select */
.register-form input,
.register-form select {
  width: 100%;
  padding: 12px 14px;
  border-radius: 8px;
  border: 1px solid #d8c7a7;
  font-size: 14px;
  background-color: #fff;
  color: #3e3e3e;
  transition: all 0.2s ease;
}

.register-form input::placeholder {
  color: #a08f77;
}

.register-form input:focus,
```

Figure 54: Register.css

```
/* Button */
.register-form button {
  margin-top: 10px;
  padding: 14px;
  border-radius: 8px;
  border: none;
  background-color: #b08968;
  color: #ffffff;
  font-size: 15px;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.25s ease, transform 0.1s ease;
}

.register-form button:hover {
  background-color: #6b4f3f;
}

.register-form button:active {
  transform: scale(0.98);
}

/* Login link below form */
.login-link {
  margin-top: 18px;
  font-size: 14px;
  color: #8a7a64;
}

.login-link a {
  color: #b08968;
  text-decoration: none;
  font-weight: 600;
}
```

Figure 55: Register.css

8.9.2 Backend

8.9.2.1 Login and register

- User Model

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    firstName: { type: String, required: true, trim: true },
    lastName: { type: String, required: true, trim: true },
    fullName: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true },
    password: { type: String, required: true },
    role: { type: String, enum: ["Freelancer", "SME", "Admin"], default: "Freelancer" },
    isEmailVerified: { type: Boolean, default: false },
    kycStatus: { type: String, enum: ["Pending", "Approved", "Rejected"], default: "Pending" },
    kycDocument: { type: String }, // file path or URL
  },
  { timestamps: true }
);

// refresh token
refreshToken: { type: String },

module.exports = mongoose.model("User", userSchema);
```

Figure 56: User model

- User Controller

```
d > controllers > JS userController.js > loginController
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");

/* REGISTER USER */
async function userController(req, res) {
  try {
    const { firstName, lastName, fullName, email, password, confirmPassword, role } = req.body;

    // BASIC VALIDATION
    if (!firstName || !lastName || !email || !password || !confirmPassword) {
      return res.status(400).json({ message: "All required fields must be filled" });
    }

    // EMAIL VALIDATION
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
      return res.status(400).json({ message: "Invalid email format" });
    }

    // PASSWORD VALIDATION
    if (password.length < 8) {
      return res.status(400).json({ message: "Password must be at least 8 characters long" });
    }
    const passwordRegex = /^(?=.*[A-Za-z])(?=.*\d)/;
    if (!passwordRegex.test(password)) {
      return res.status(400).json({ message: "Password must contain at least one letter and one number" });
    }
    if (password !== confirmPassword) {
      return res.status(400).json({ message: "Passwords do not match" });
    }

    // ROLE PROTECTION
```

Figure 57: User controller

```
async function userController(req, res) {  
  // CHECK USER  
  const existingUser = await User.findOne({ email });  
  if (existingUser) {  
    return res.status(400).json({ message: "Email already registered" });  
  }  
  
  // HASH PASSWORD  
  const hashedPassword = await bcrypt.hash(password, 10);  
  
  // COMPUTE FULL NAME  
  const computedFullName = fullName && fullName.trim() !== "" ? fullName : `${firstName} ${lastName}`;  
  
  // CREATE USER  
  const user = await User.create({  
    firstName,  
    lastName,  
    fullName: computedFullName,  
    email,  
    password: hashedPassword,  
    role: finalRole,  
  });  
  
  // JWT TOKENS  
  const accessToken = jwt.sign(  
    { userId: user._id, role: user.role },  
    process.env.JWT_SECRET,  
    { expiresIn: "15m" } // short-lived token  
  );  
  
  const refreshToken = jwt.sign(  
    { userId: user._id, role: user.role },  
    process.env.JWT_REFRESH_SECRET,  
  );  
}
```

Figure 58: User controller

```
// Save refresh token in DB
user.refreshToken = refreshToken;
await user.save();

res.status(201).json({
  message: "User registered successfully",
  accessToken,
  refreshToken,
  user: {
    id: user._id,
    email: user.email,
    role: user.role,
  },
});
} catch (error) {
  console.error("Register error:", error.message);
  res.status(500).json({ message: error.message });
}
}

/* LOGIN USER */
async function loginController(req, res) {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: "Email and password are required" });
    }

    // FIND USER
    const user = await User.findOne({ email });
    if (!user) {
```

Figure 59: User Controller

```
// COMPARE PASSWORD
const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) {
  return res.status(400).json({ message: "Invalid email or password" });
}

// JWT TOKENS
const accessToken = jwt.sign(
  { userId: user._id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: "15m" }
);

const refreshToken = jwt.sign(
  { userId: user._id, role: user.role },
  process.env.JWT_REFRESH_SECRET,
  { expiresIn: "7d" }
);

// Save refresh token in DB
user.refreshToken = refreshToken;
await user.save();

res.status(200).json({
  message: "Login successful",
  accessToken,
  refreshToken,
  user: {
    id: user._id,
    email: user.email,
    role: user.role,
  },
});
```

Figure 60: User Controller

- Auth Middleware

```
const jwt = require("jsonwebtoken");

const authMiddleware = (req, res, next) => {
  const authHeader = req.headers.authorization;

  // Check token exists
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Access denied. No token provided." });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { userId, role }
    next();
  } catch (error) {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
};

module.exports = authMiddleware;
```

Figure 61: Auth Middleware

- Auth Route

```
const express = require("express");
const router = express.Router();

const {
  userController,
  loginController,
} = require("../controllers/userController");

const authMiddleware = require("../middleware/authMiddleware");
const User = require("../models/User");
const jwt = require("jsonwebtoken");

// PUBLIC ROUTES
router.post("/register", userController);
router.post("/login", loginController);

/* REFRESH TOKEN */
router.post("/refresh", async (req, res) => {
  const { token: refreshToken } = req.body;

  if (!refreshToken) {
    return res.status(401).json({ message: "Refresh token required" });
  }

  try {
    // Verify refresh token
    const decoded = jwt.verify(refreshToken, process.env.JWT_REFRESH_SECRET);

    // Find user and match refresh token
    const user = await User.findById(decoded.userId);
    if (!user || user.refreshToken !== refreshToken) {
      return res.status(403).json({ message: "Invalid refresh token" });
    }
  }
});
```

Figure 62: Auth Route

```

router.post("/refresh", async (req, res) => {

  // Issue new access token
  const accessToken = jwt.sign(
    { userId: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "15m" }
  );

  res.status(200).json({ accessToken });
} catch (err) {
  console.error("Refresh token error:", err);
  res.status(403).json({ message: "Invalid or expired refresh token" });
}
});

// PROTECTED ROUTE
router.get("/profile", authMiddleware, (req, res) => {
  res.json({
    message: "Profile accessed",
    user: req.user, // decoded from JWT
  });
});

module.exports = router;

```

Figure 63: Auth Route

- Config

```

const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
  }
};

module.exports = connectDB;

```

Figure 64: Config

8.9.2.2 Sprint Review

At the conclusion of Sprint 1, a review session was held to showcase the implemented login and registration features. The functionality was demonstrated to ensure it aligned with the sprint objectives and specified requirements.

Observations and feedback related to system flow, input validation, and user interaction were collected, helping confirm completion while identifying small refinements for future development.

8.9.2.3 Sprint retrospective

The retrospective involved reflecting on the sprint's execution to understand both strengths and difficulties. The team discussed effective practices as well as obstacles faced during design and development. Based on these discussions, improvement strategies were outlined to enhance planning, coordination, and execution in the next sprint.

8.10 Progress Analysis

8.10.1 Analysis

During Sprint 1, both frontend and backend development tasks were completed as planned. Essential system features, including user registration, login functionality, and authentication mechanisms, were implemented and reviewed. Sprint review and retrospective sessions confirmed that objectives were met, while velocity and burndown charts reflected effective sprint planning and task management.

Sprint 2 planning is currently underway. The consistent progress achieved so far indicates readiness to advance toward more complex system features in upcoming sprints. Overall, the project remains on schedule, demonstrates steady development progress, and continues to align closely with the defined project objectives.

8.11 Future Works

8.11.1 Sprint 2: Project Posting and Bidding Module

Sprint Objective:

The objective of this sprint is to develop the essential marketplace features of the platform by facilitating interaction between SMEs (clients) and freelancers. This sprint serves as the base layer upon which all subsequent system functionalities are built.

Functional Scope:

- SMEs can post, update, and manage project details
- Freelancers can explore available projects and submit bids or proposals
- Projects can be matched and filtered based on required skills
- SMEs can review, compare, and select suitable proposals

Key Features:

Project Posting

- Project name, detailed description, and required skill set
- Budget specification (fixed-price or hourly basis)
- Defined deliverables and expected completion dates

Project Listing & Search

- Browsing projects by category
- Filtering options based on budget, skills, and deadlines

Bidding/Proposal System

- Submission of proposals along with a cover message
- Quoted price and estimated completion duration
- Visibility of freelancer profiles and portfolios

Tools & Technologies

- Frontend: React.js
- Backend: Node.js with Express
- Database: MongoDB
- APIs: RESTful services for managing project-related operations
- Authentication: JWT-based user authentication
- Version Control: Git with GitHub or GitLab

8.11.2 Sprint 3: Escrow and Payment Management

Sprint Objective:

This sprint aims to implement a secure payment handling mechanism that builds trust between SMEs and freelancers by holding payments safely until project completion.

Functional Scope:

- Secure payment holding through escrow
- Payment release after project approval
- Payment status tracking
- Transaction history management

Key Features:

Escrow Workflow

- SMEs deposit project payments into escrow
- Payments are released after final project approval

Payment Tracking

- Viewing pending, completed, and refunded payments

Tools & Technologies

- Payment Gateway: esewa (sandbox environment)
- Backend: Secure payment APIs built with Express.js
- Security: HTTPS, SSL/TLS encryption
- Database: MongoDB collections for transactions and escrow records
- Logging: Activity logs for financial operations

8.11.3 Sprint 4: Communication, File Sharing, and Review System

Sprint Objective:

The purpose of this sprint is to enhance collaboration, transparency, and credibility by introducing internal communication, file sharing, and user feedback features.

Functional Scope:

- Internal messaging between SMEs and freelancers

- Secure upload and download of project files
- Rating and review system

Key Features

Messaging System

- One-to-one real-time chat
- Message notifications

File Sharing

- Uploading and downloading project documents and deliverables
- Validation of file types and sizes

Review & Rating System

- Star-based ratings
- Written feedback after project completion
- Freelancer reputation scoring

Tools & Technologies

- File Storage: Cloud storage (e.g., AWS S3) or local server
- Frontend: React-based chat interface
- Backend: Secure file handling APIs using Node.js
- Database: MongoDB collections for messages, files, and reviews
- Security: Access control and file validation

8.11.4 Sprint 5: Administrative Dashboard and KYC Verification

Sprint Objective:

This sprint focuses on improving system security and governance by providing administrative controls and implementing user identity verification.

Functional Scope:

- Admin dashboard for system monitoring

- User, project, and transaction management
- KYC document verification

Key Features:

Admin Dashboard

- User account activation and suspension
- Project monitoring and moderation
- Overview of platform transactions

KYC Verification

- Manual verification by administrators

Fraud Prevention

- Detection and flagging of suspicious user activity
- Tracking KYC verification status

Tools & Technologies

- Frontend: React.js (Admin Panel)
- Backend: Node.js with role-based access control
- Database: MongoDB collections for KYC data and admin logs
- Document Processing: Basic validation
- Security: Two-factor authentication (2FA)

8.11.5 Sprint 6: AI-Based Cost and Timeline Estimation

Sprint Objective:

The goal of this sprint is to introduce intelligent features that assist users by providing estimated project costs and timelines based on historical data.

Functional Scope:

- Analysis of previous project data
- Estimation of project cost and duration

- Recommendation support during project creation

Key Features:

- Cost Estimation
- Estimates generated using project scope and complexity
- Timeline Prediction
- Duration predictions based on similar completed projects
- Freelancer Matching Support
- Skill and experience-based recommendation suggestions

Tools & Technologies

- Machine Learning: Python, Scikit-learn
- Data Processing: Pandas, NumPy
- Model Types: Regression models
- Backend Integration: REST APIs connecting ML services to MERN backend
- Data Storage: Historical project data in MongoDB

8.11.6 Sprint 7: System Integration, Testing, and Optimization

The final sprint will focus on integrating all modules into a unified system. Comprehensive functional, security, and performance testing will be conducted to identify and resolve issues. System optimization and final documentation will be completed to ensure readiness for deployment and evaluation.

